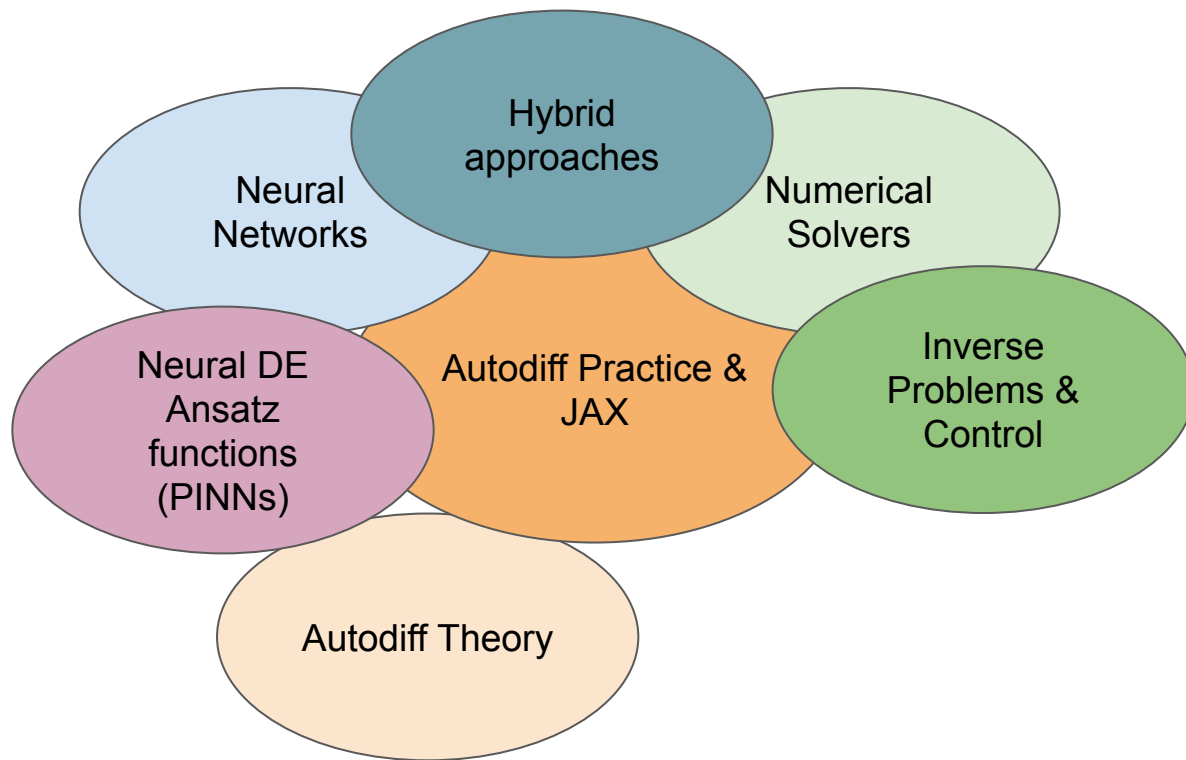


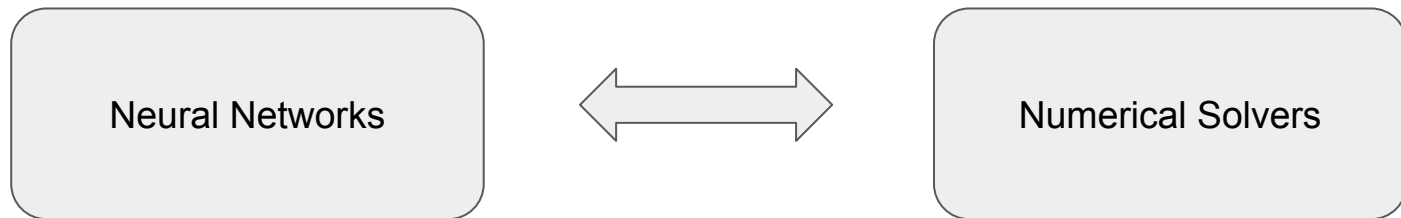
The Hybridization of Solvers and Deep Learning with Differentiable Physics

Felix Koehler
ML & AD in JAX
Strasbourg, France
11 June 2026





Marrying NNs & Numerical Solvers



1. Which parameters are being optimized?
2. What are the primitive operations?
3. At which granularity level of AD?
4. What are the benefits of each and how can they be combined?

Seeing this makes the hybridization approaches pretty obvious!

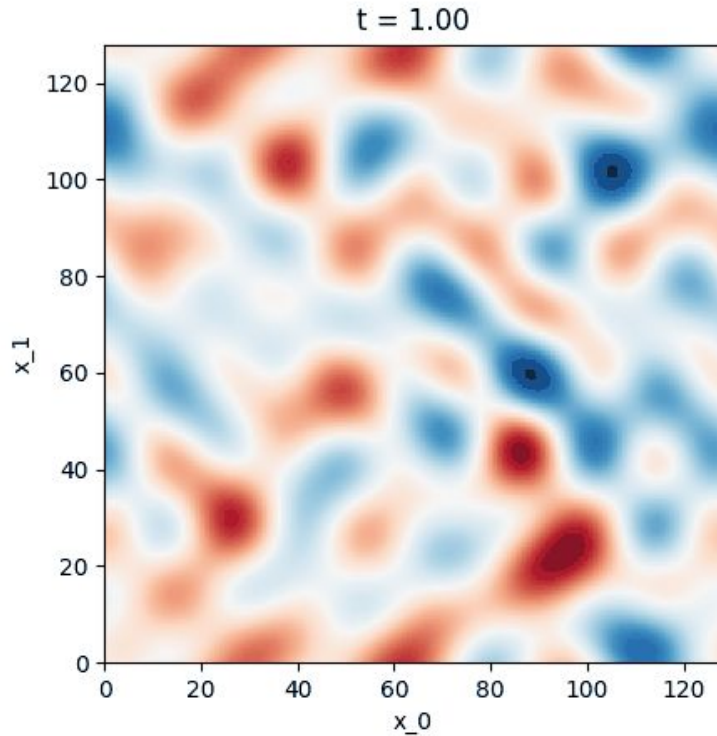
Agenda

1. Numerical Simulators vs. Neural Networks
2. Hybridization and its look-a-alikes
3. Why and how to hybridize (with full-field correction approaches)
4. Nuances and theoretical insight
5. Concrete Case Studies
6. Practical Session Preparation

Please ask questions
during the talk!

Numerical Solvers vs. Neural Networks

Animation of 2D Advection

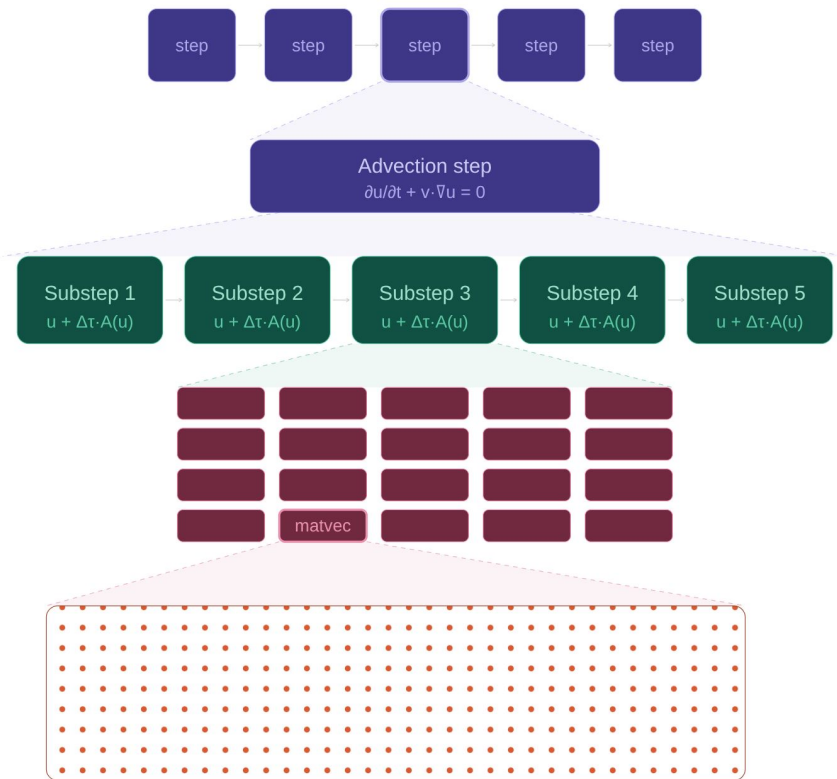


$$\partial_t u + \partial_x u + \partial_y u = 0$$

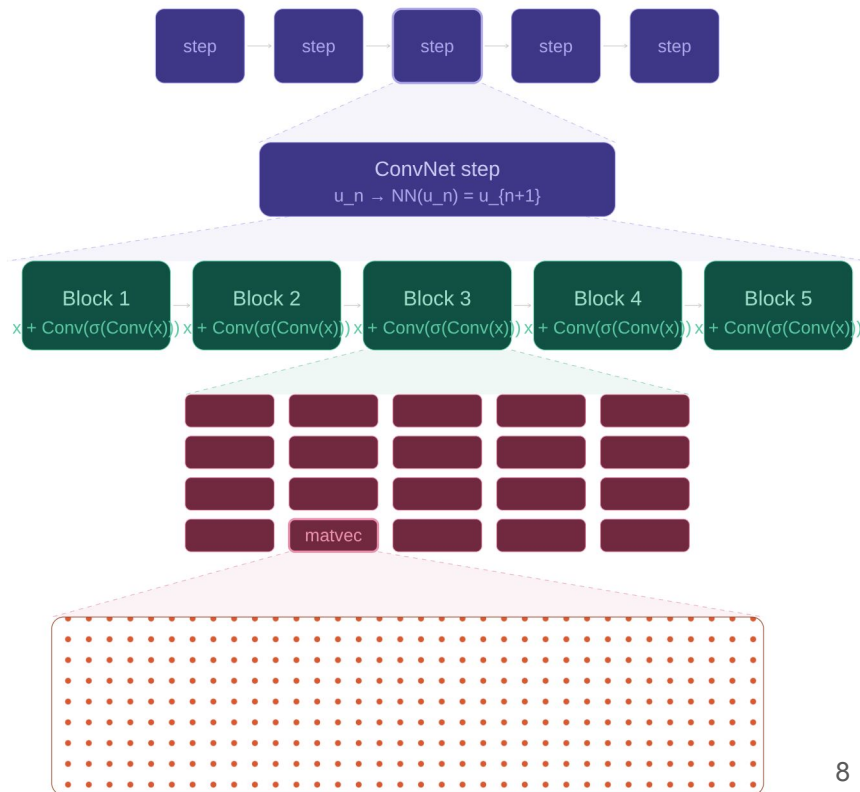
$$u(t, x = 0, y) = u(t, x = L, y)$$

$$u(t, x, y = 0) = u(t, x, y = L)$$

Compute Graphs

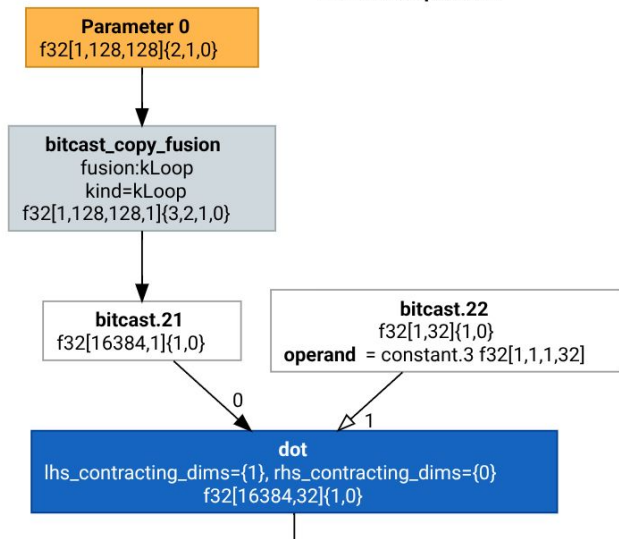


You may want to check out
<https://github.com/zombie-einstein/jaxpr-viz>

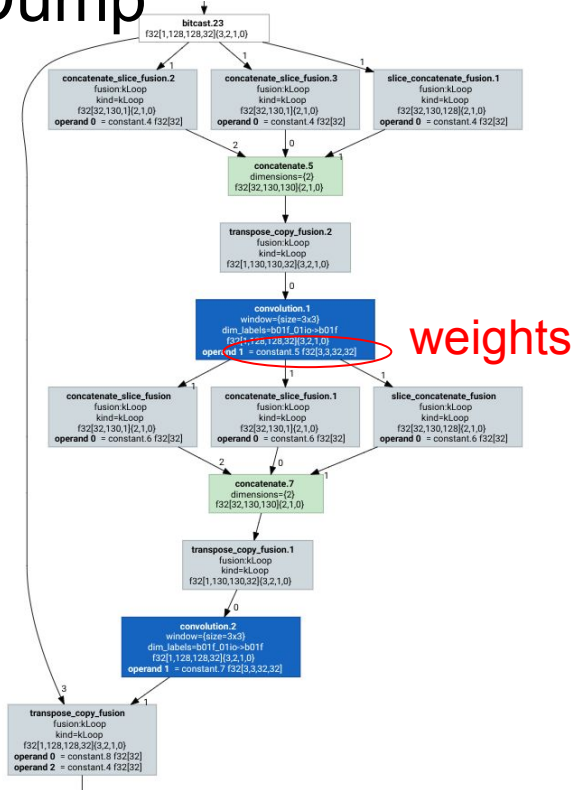


ConvNet HLO Graph Dump

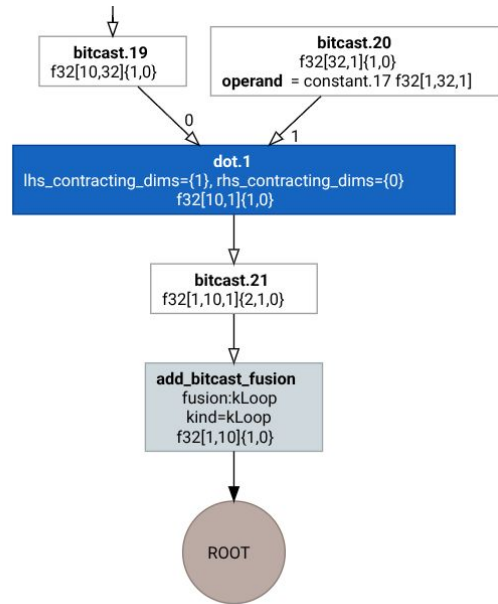
module_0027.jit_apply_conv_net.cpu_after_optimizations
Computation main.3
ENTRY computation



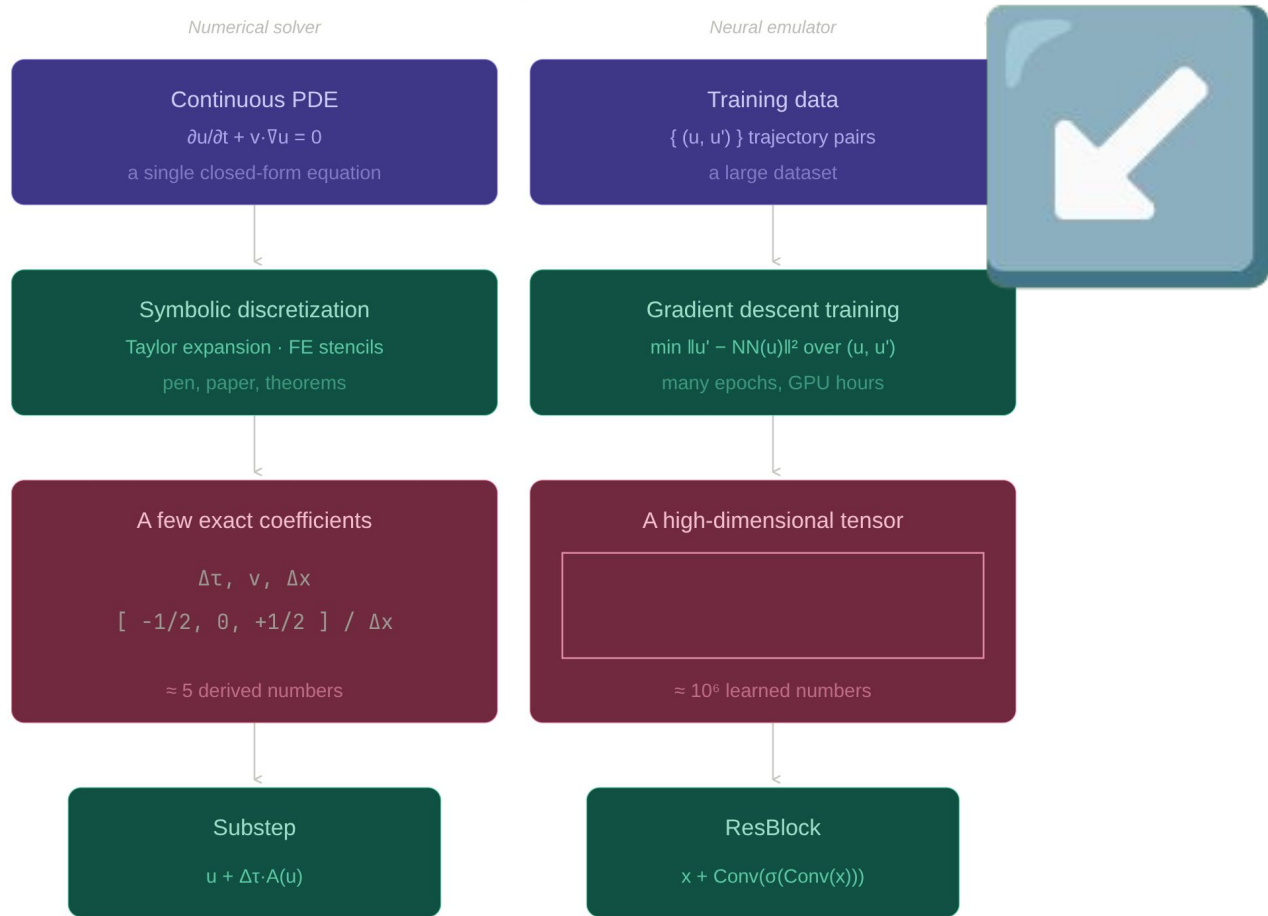
Lifting



Processing



Projection



Autodiff is straightforward for **explicit** operations!

- Floating point operations ✓
- Matrix Multiplication and Addition ✓
- Convolution ✓
- Pointwise nonlinearity ✓
- Broadcasting/Reductions ✓
- Array Manipulations ✓

$$z = x + y$$

$$\bar{x} = \bar{z}$$

$$\bar{y} = \bar{z}$$

$$z = x \cdot y$$

$$\bar{x} = \bar{z} \cdot y$$

$$\bar{y} = \bar{z} \cdot x$$

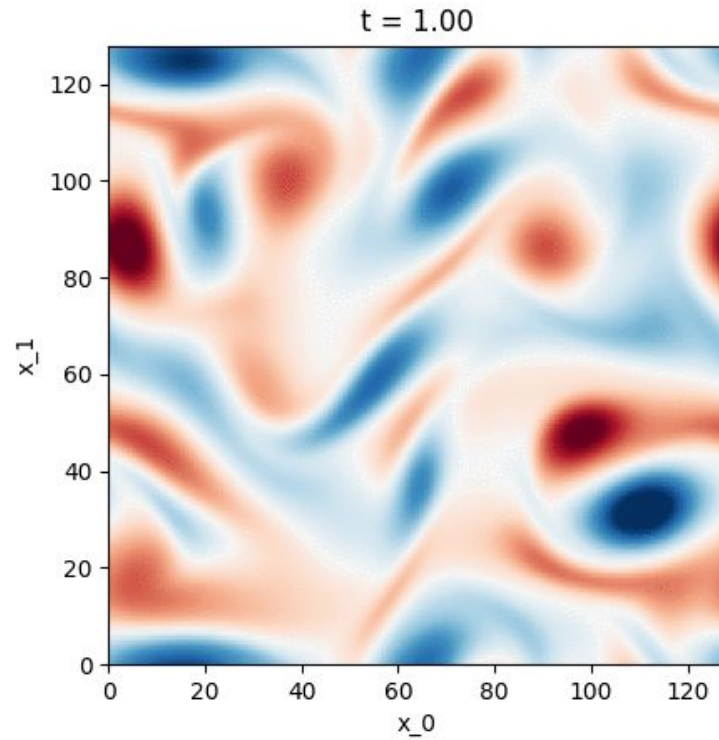
$$\mathbf{C} = \mathbf{A}\mathbf{B}$$

$$\bar{\mathbf{A}} = \bar{\mathbf{C}}\mathbf{B}^T$$

$$\bar{\mathbf{B}} = \mathbf{A}^T\bar{\mathbf{C}}$$

<https://fkoehler.site/autodiff-table/>

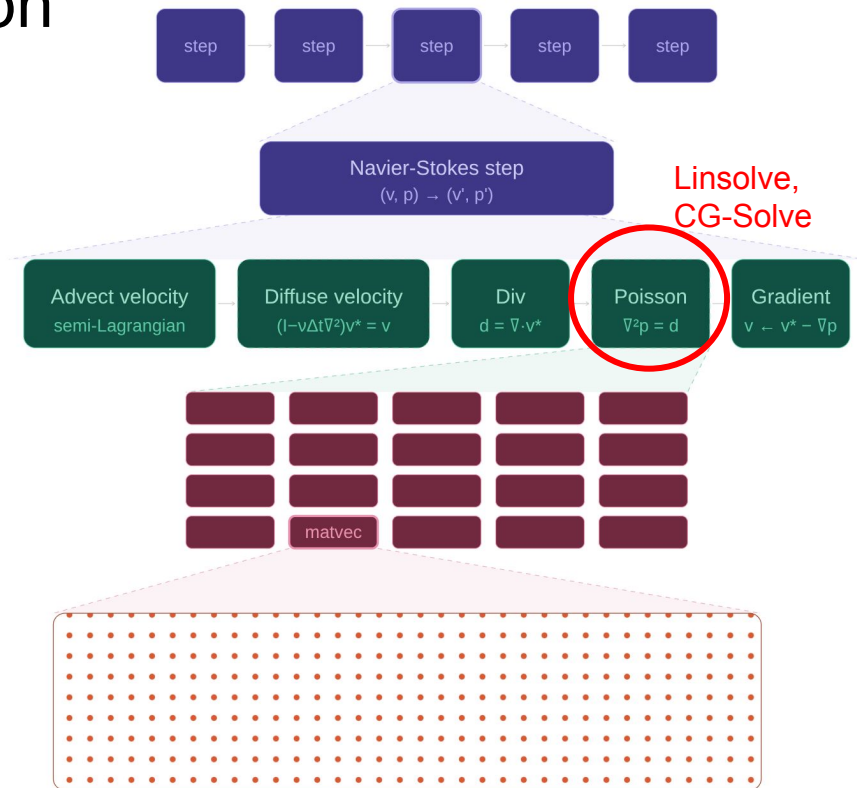
Kolmogorov Flow in 2D



Navier-Stokes Compute Graph

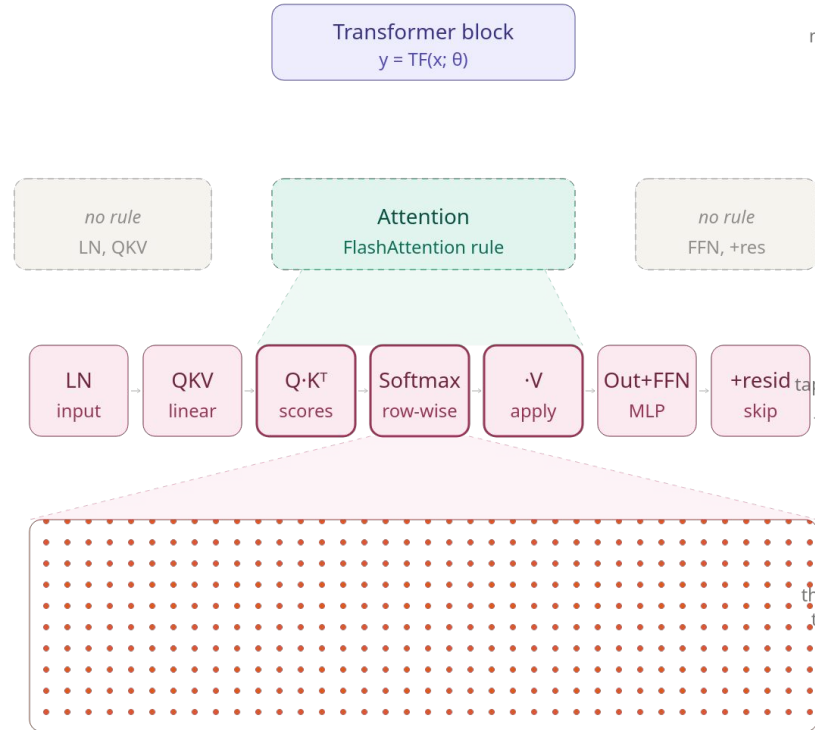
Reveals another level of granularity in the compute graph

- With the custom roots implicit differentiation in JAX
- Kind of finding a higher-level primitive to avoid algorithmic unrolling

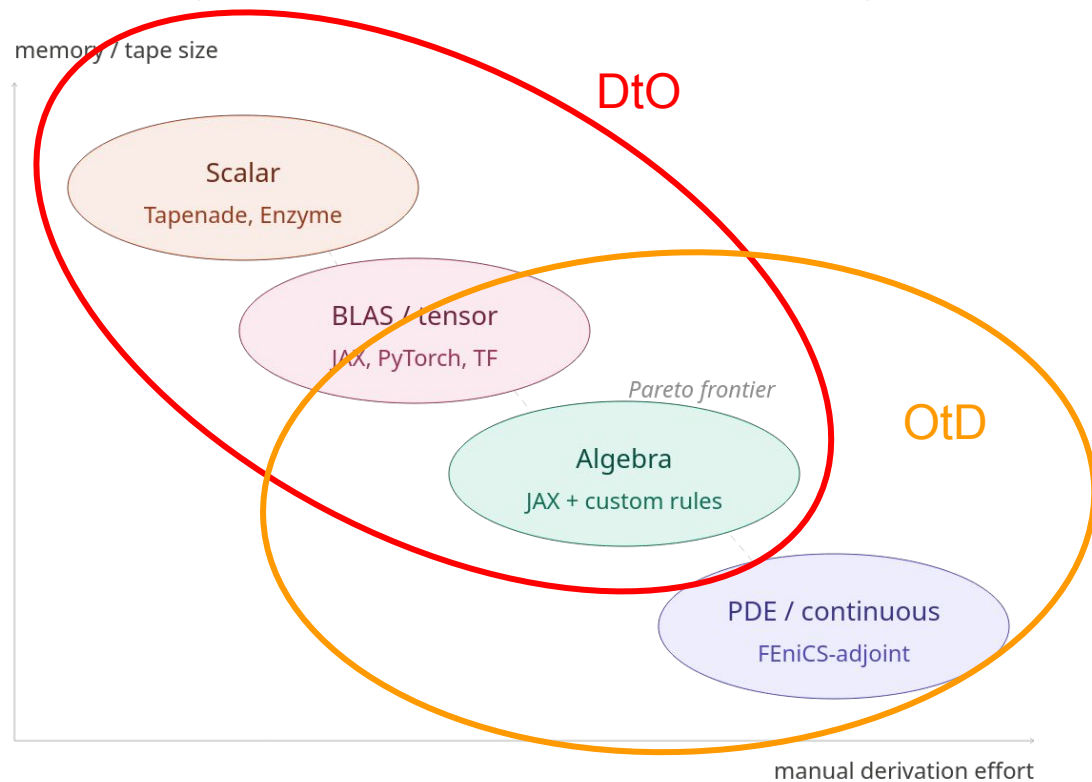


<https://fkoehler.site/implicit-autodiff-table/>

We also see higher level primitives in neural nets



Trading automatability and compute/memory cost



Autodiff Levels across Compute Graphs

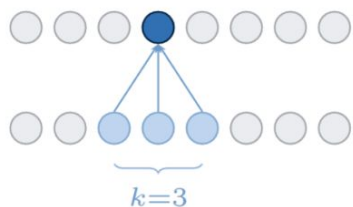
	PDE solver	MLP / ConvNet	Transformer	Gaussian Splatting
Operation 1 atomic op	Solve PDE manual adjoint	MLP no analytic adj.	TF block no analytic adj.	
Algebra implicit rules	Linsolve implicit rule	<i>no rule skipped</i>	Attention FlashAttn rule	
BLAS tape tensors	matvec, dot axpy, scale tens of ops	Linear, ReLU elementwise few ops	QKV, softmax LN, FFN, +res several ops	
Scalar tape floats	+, ×, sin, exp	+, ×, exp	+, ×, exp	

Benefits of numerical solvers and neural networks

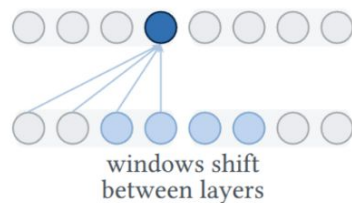
- Numerical solvers have guarantees:
 - “Weights” are derived from symbolic manipulations, discretization approaches, and first principles:
 - Parameters are exact floating point values up to machine precision
 - Neural Network weights are found as part of a (stochastic) optimization process. They are fuzzy, many minima exist, and the networks are not converged to machine precision
 - We also know how and when they tend to fail
 - We have principled ways to make them more accurate.
- Neural Networks can:
 - Process different modalities of data
 - Neural Networks tend to avoid implicit solvers: everything is kept explicit
 - Are fuzzy and may bypass numerical constraints
 - Are more pragmatic?
 - Are easier to get started with 😊
 - Are made for GPUs!

Surprising Similarity for Spatial Processing - Or Not?

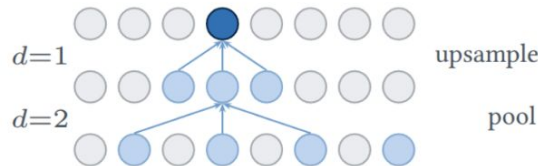
(a) ConvNet



(b) SWIN



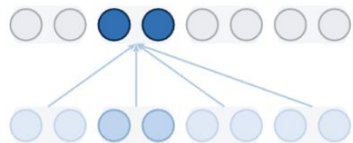
(c) Dilated ResNet



(d) UNet



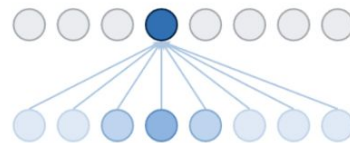
(e) ViT



(f) FNO



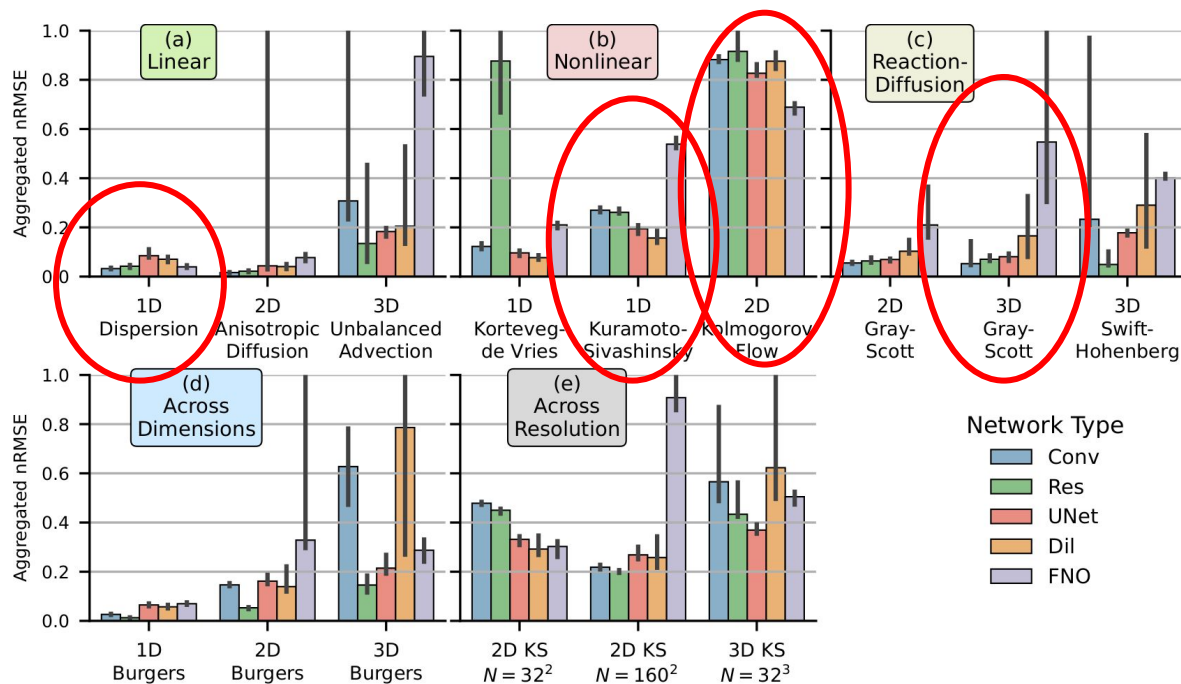
(g) Transformer



Legend

- output (target)
- strong contribution
- moderate contribution
- weak contribution
- inactive position
- information flow
- frequency mode

What this similarity between neural architectures and numerical simulators means for purely predictive neural emulators



Summary

- Simple numerical solvers (substepped explicit solvers) have an explicit compute graph and tensor operations like typical neural networks
- More sophisticated simulators/PDEs introduce implicit solvers:
 - Less common in neural networks (though sometimes used like Deep Equilibrium Models)
 - Differentiating them often with implicit differentiation
 - Manual optimizations in the backward pass also common in learned approaches
- While similar in functional form:
 - Simulators derived from first principles, have guarantees, require domain knowledge, and often have limitations
 - NNs use a data-driven optimization, can combine different sources of data, are flexible, and often slightly better suited for compute accelerators (GPU, TPU, ...)
 - The similar functional form often correlates with performance as emulators

Hybridization and its look-alikes

Hybridization

=

A neural network and
a numerical solver*
working together

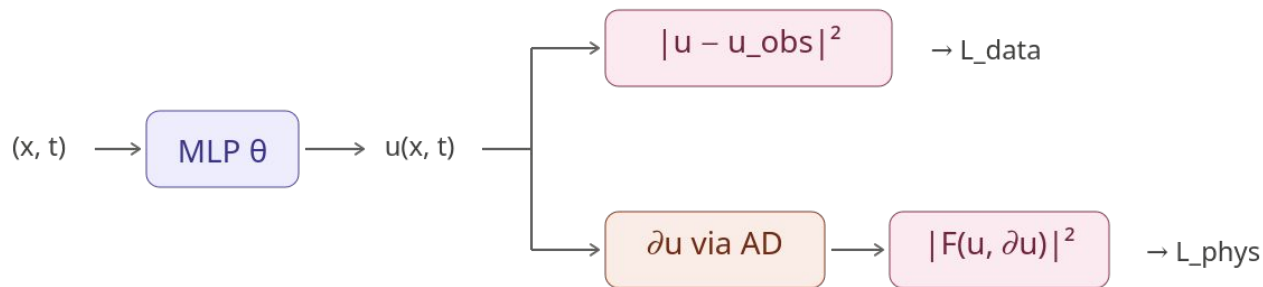
Differentiable Physics

=

A numerical solver for
a “physical” ODE/PDE
in the backward pass

That does not mean PINNs

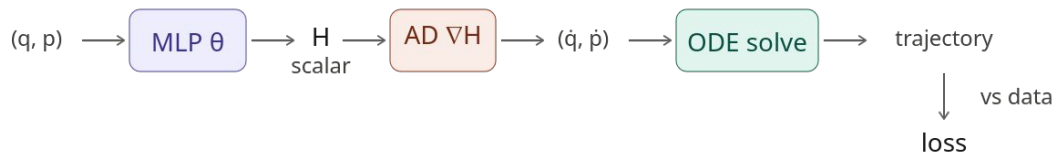
Because they prescribe a constraint and do not use a solver



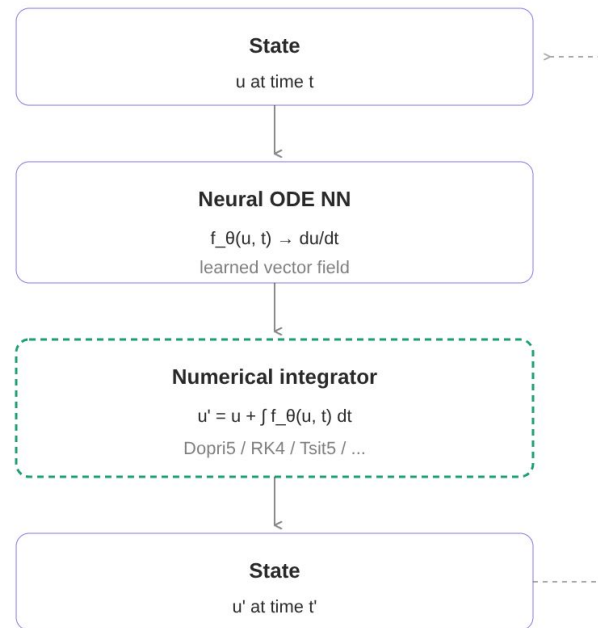
That does not mean Neural ODEs, HNNs or LNNs

Because they do not have a hand-derived discretization for a “specific” ODE/PDE in them

(though they are nice showcases of autodiff)



$$\text{where } \dot{q} = \partial H / \partial p, \dot{p} = -\partial H / \partial q$$



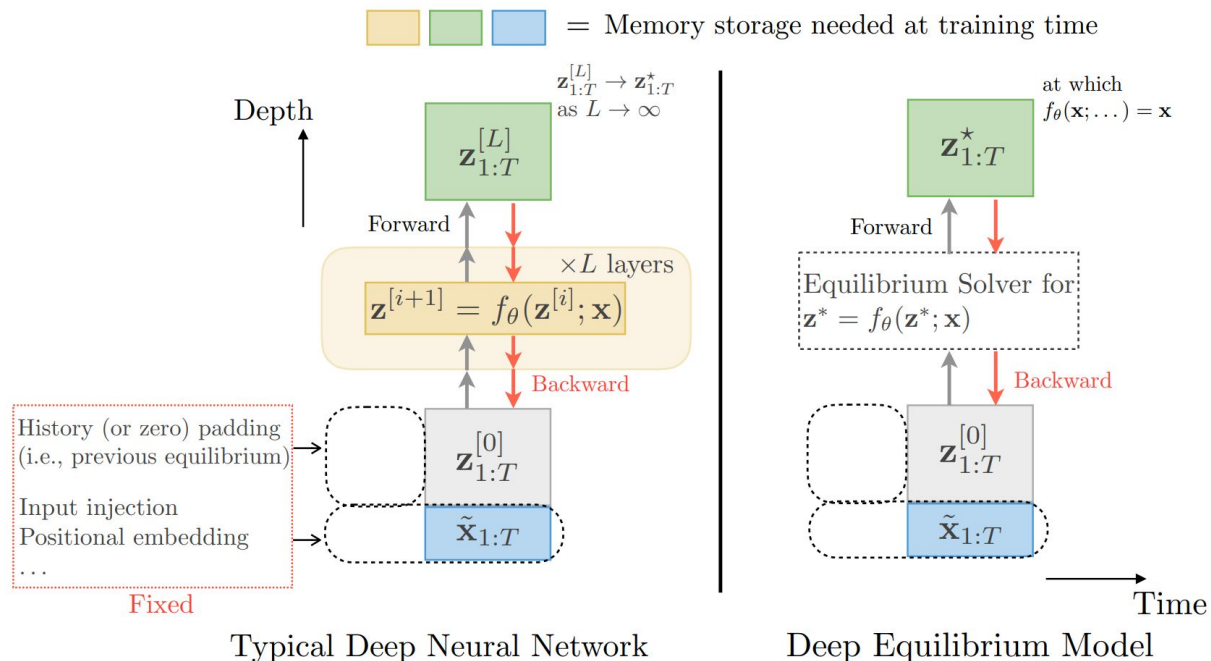
Chen, Ricky TQ, et al. "Neural ordinary differential equations." *Advances in neural information processing systems* 31 (2018).

Greydanus, Samuel, Misko Dzamba, and Jason Yosinski. "Hamiltonian neural networks." *Advances in neural information processing systems* 32 (2019).

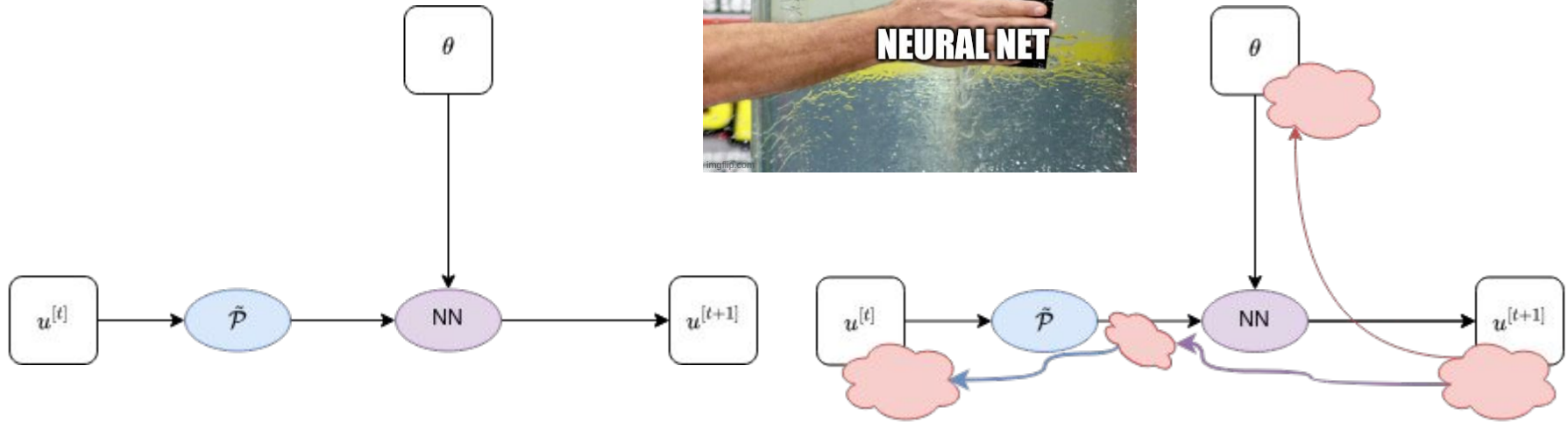
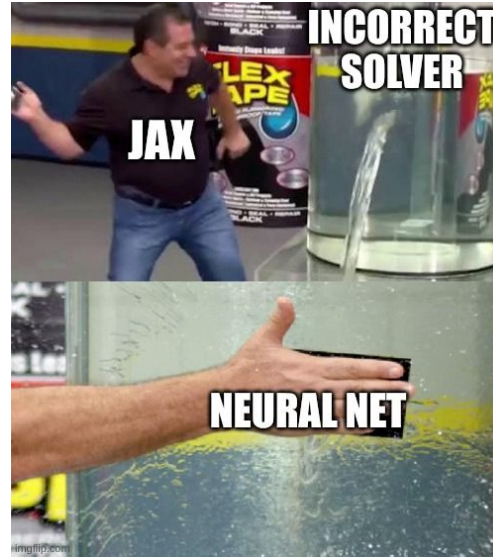
Cranmer, Miles, et al. "Lagrangian neural networks." *arXiv preprint arXiv:2003.04630* (2020).

That also does not mean DEQs, Implicit Layers etc.

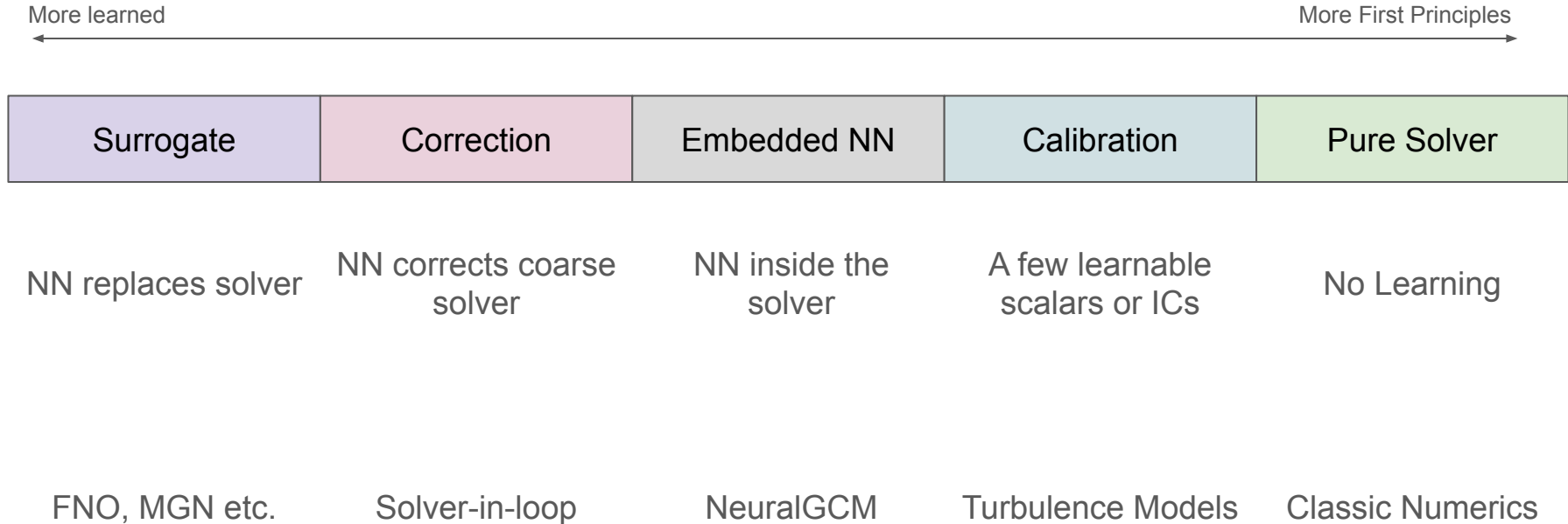
Because they apply a root-finder to a neural condition and do not involve concrete “physics”



Sequential Correction



Allowing for a range of what “neural networks” could be reveals a depth of integration for Learned Components

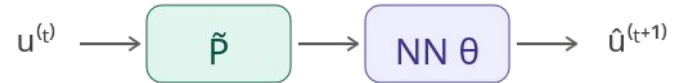


What is going on at the different approaches?

Purely Neural Prediction



Full-Field Correction



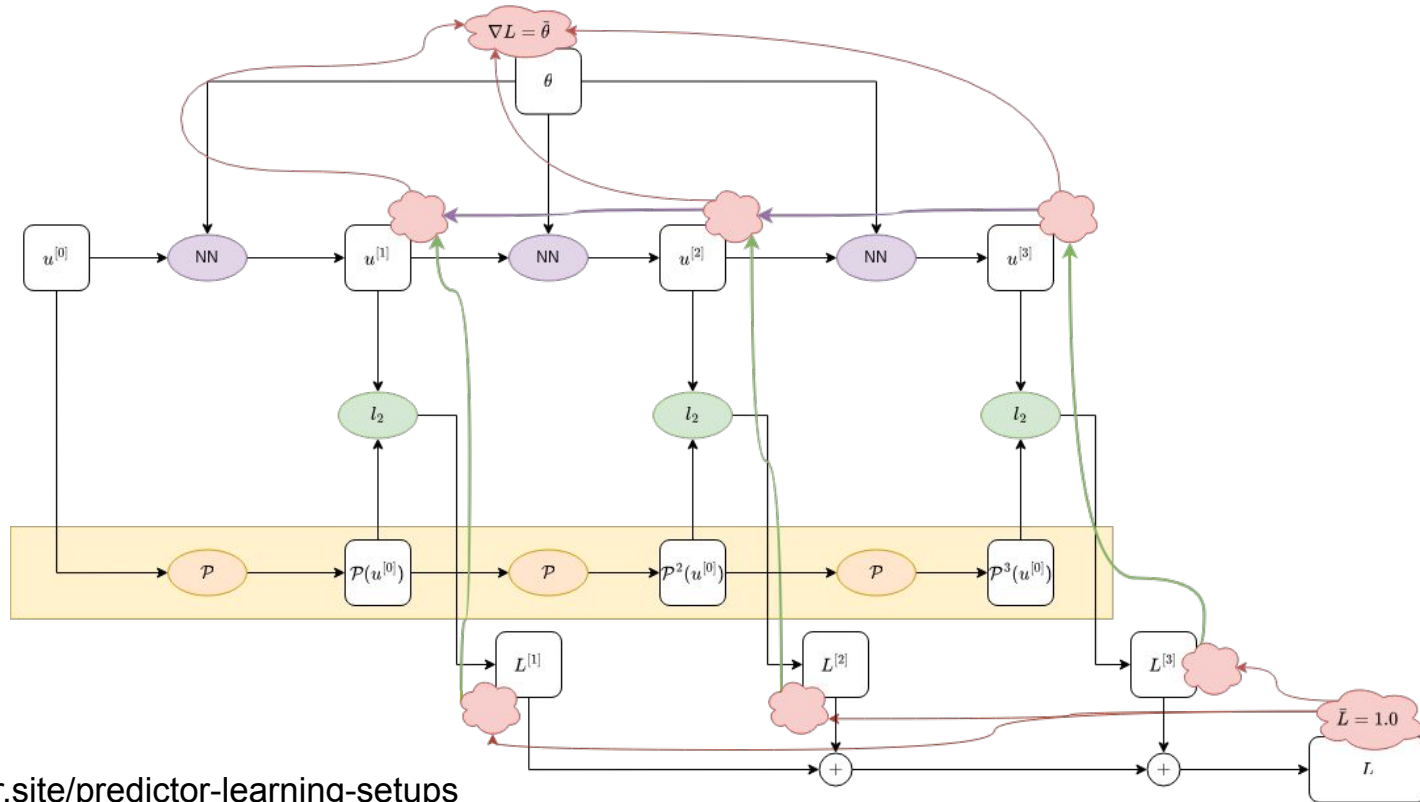
Part Modelling



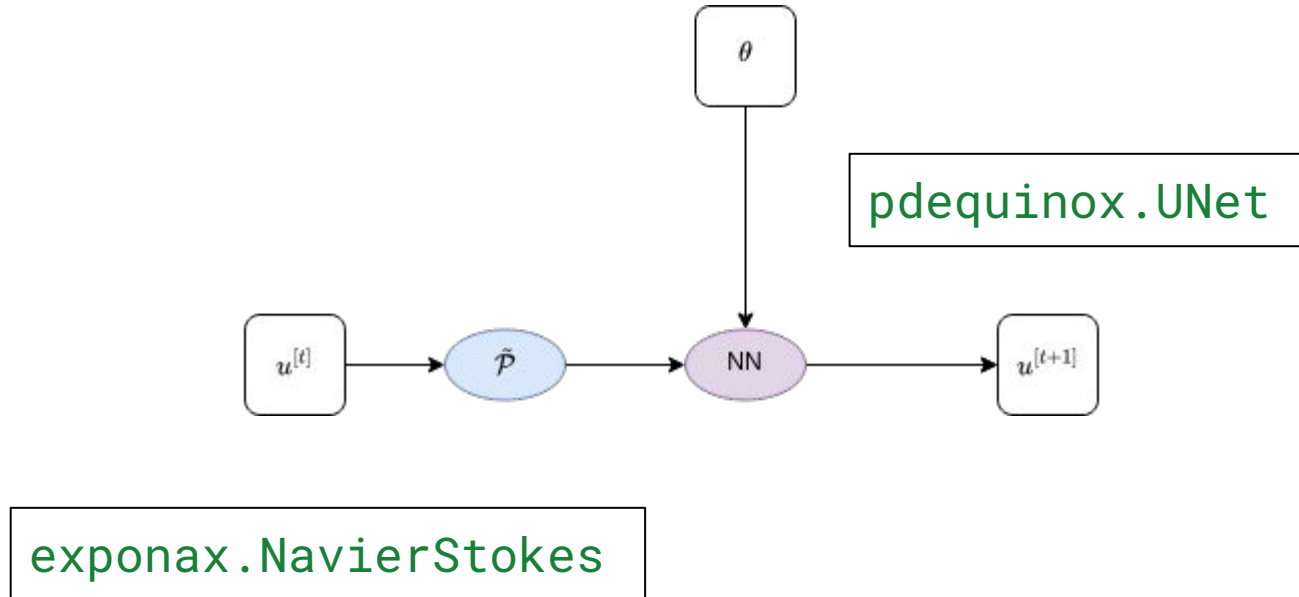
Calibration, Data Assimilation, ...



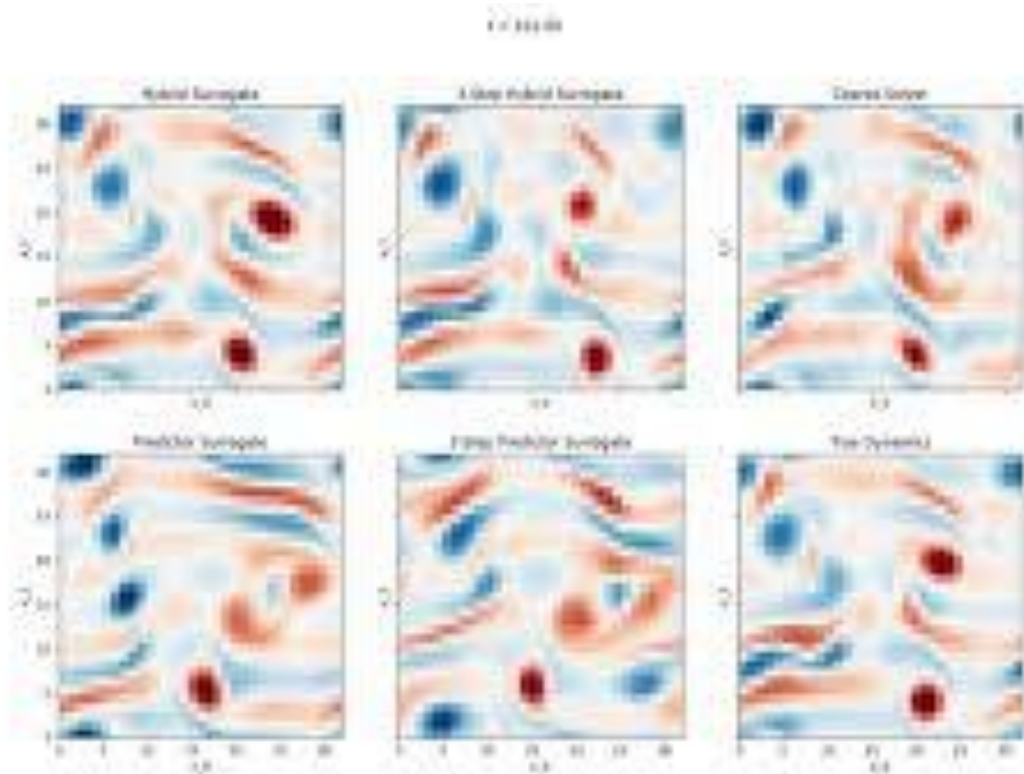
The backpropagation through time



For the practical exercise



Numerical Solvers replaced by learned hybrid emulator

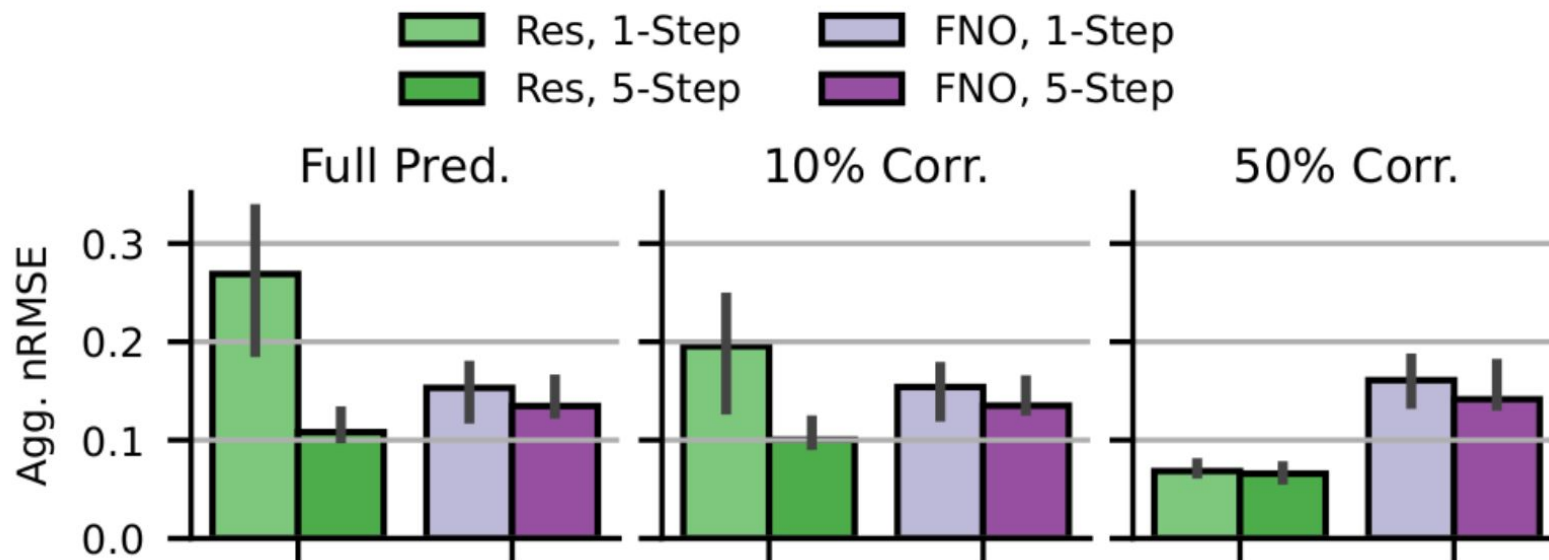


Summary

- The progress in autodiff tooling in the second half of 2010s spawned many interesting research directions (tangential to this talk):
 - (input-output autodiff) residuum-based penalizers or PINNs
 - Hierarchical autodiff for Hamiltonian and Lagrangian NNs
 - Autodiff through temporal integrators in Neural ODEs
 - Autodiff through implicit solvers in DEQs
- This talk: Hybridization means using solver for concrete PDE/ODE in the loop
- Differentiable Physics can be invoked in a spectrum of applications:
 - Constitutive parameter learning and Data Assimilation
 - Learning small components
 - **Full-field correction**
- For time-dependent rollouts, backpropagation-through-time occurs
 - Both a curse and a blessing (as we shall see later)

Why and How to hybridize (for full-field correction)

APEBench: Spectral Method + ResNet better than FNO



Anecdotaly: Combine the merits of different paradigms

- Since Neural Networks mirror numerical schemes, this may bring together different paradigms
 - Like spectral methods and finite differences
 - This is also something we may find in numerical solvers
 - Linear Solvers: Multigrid with Smoothers as Krylov preconditioners
- Bring together the aforementioned strict numerical guarantees of solvers with the learning-based freedom of neural networks:
 - i.e., we can tailor to specific manifolds of the state space
 - IMO: This is also the big advantage and argument in favor of neural surrogates/emulators in the first place

Differentiating non-smooth operations

- Flux Limiters (`jnp.where` operations)
- Contact resolution in structural mechanics

.... -> SoftJAX softening certain operations

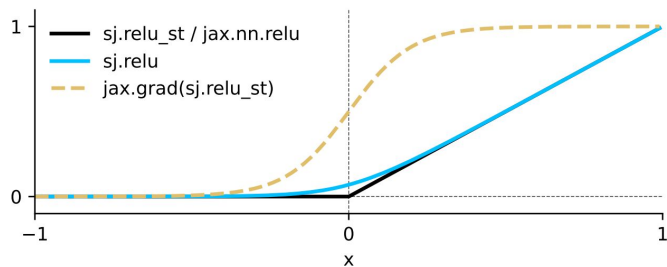
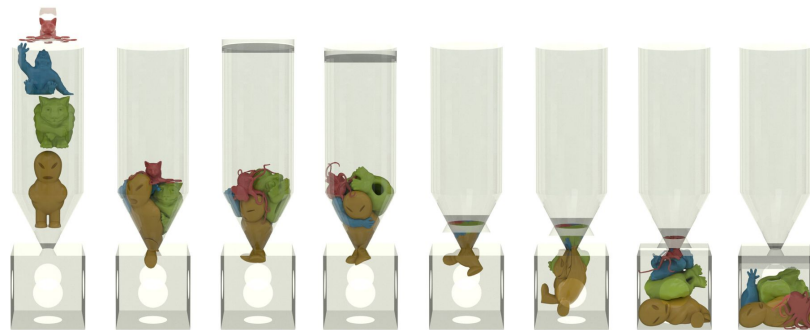


Figure 2. The function `relu_st` resorts to the straight-through trick to use the hard function `jax.nn.relu` in the forward pass while using the soft function `sj.relu` for gradient computation.

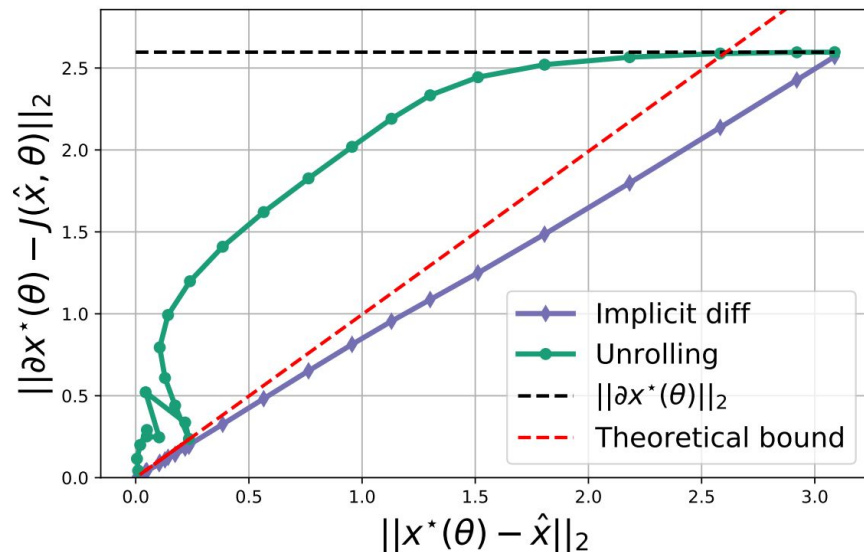


Automated and modular implicit differentiation

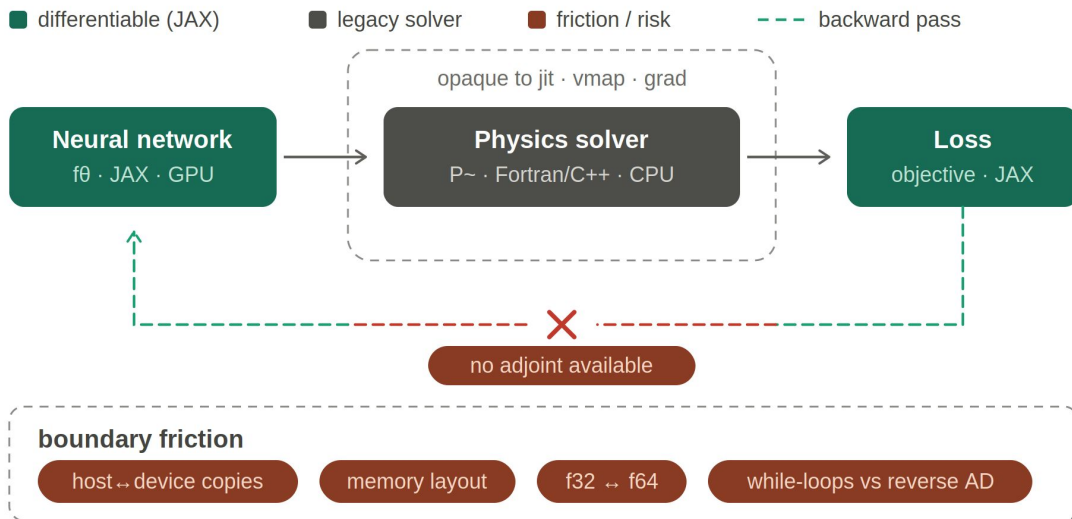
`jax.lax.custom_root`

`jax.lax.custom_linear_solve`

- Given optimality condition: automatically register autodiff rules
- Also allows for fusing a vJp
- Internally performs matrix-free linear solves



Pragmatic problems when using differentiable physics



three ways to get a gradient

Bridge the AD

AD the native source code
custom_vjp + Tesseract
exact grads · setup cost

Port to JAX

rewrite the solver in XLA
native jit, vmap, scan
NN corrects → relax rigor

Cut the gradient

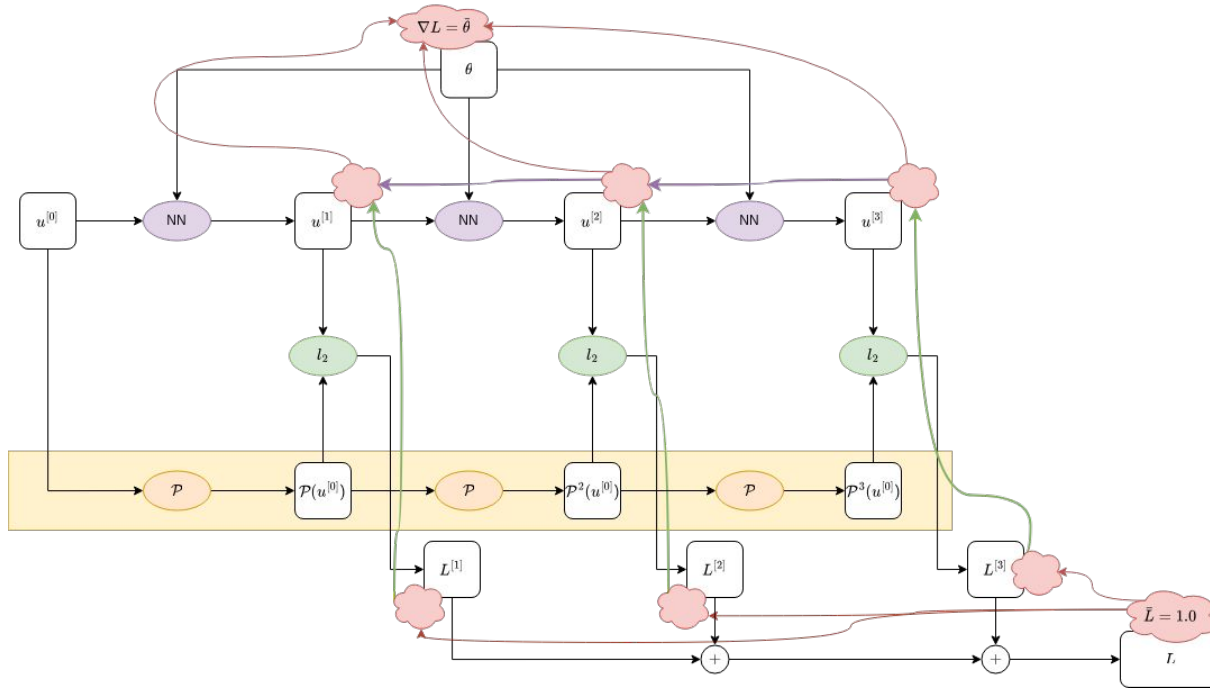
stop-grad unrolling (NOG)
or FD / ES estimates
cheap but biased grads

Summary

- Full-Field Correction approaches allow for a “deep-learning native” way of combining differentiable simulators and neural networks
 - Especially when combining different paradigms, this renders beneficial for neural surrogates
- Differentiating numerical solvers can be non-straightforward
 - As such: Whether to use differentiable physics or not might depend on the effort needed to spend here to make it “meaningfully differentiable”
 - DP could technically also be done with finite differences
 - Sanity-check solver gradients first: finite-difference/Taylor test + adjoint dot-product test
- Modern AD tooling allow for better differentiable interoperability between various components
 - But it can still be tedious!

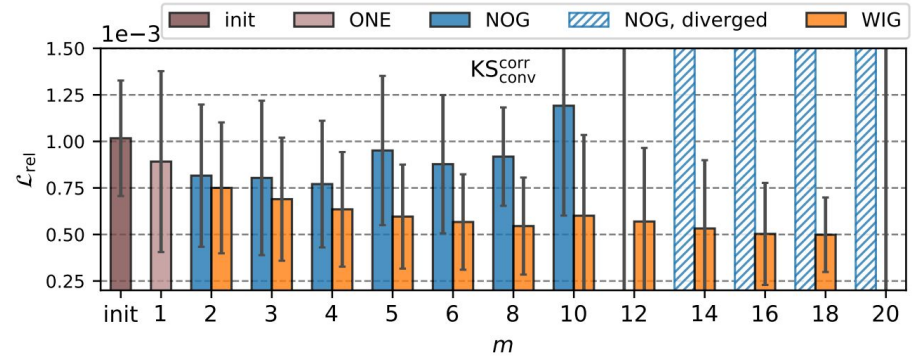
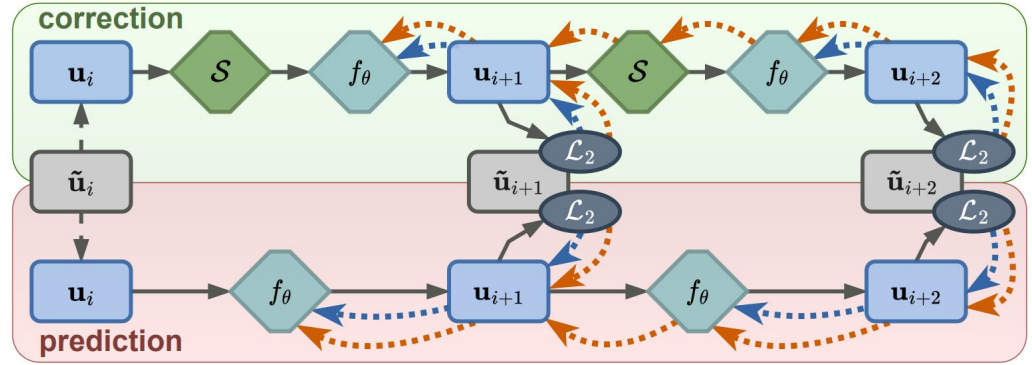
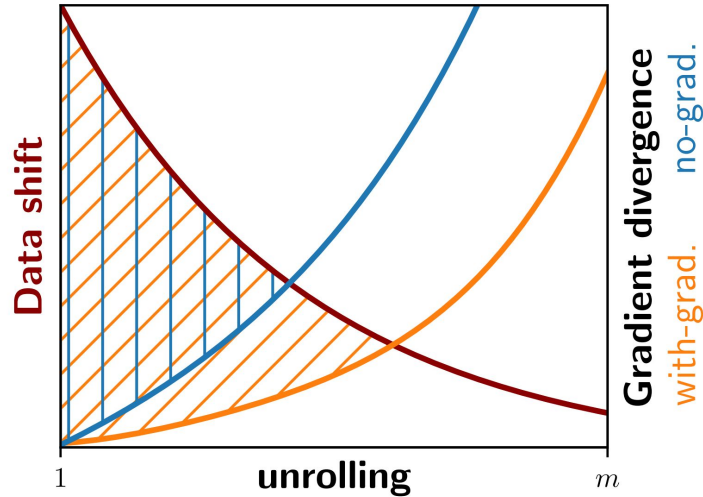
Nuances of Hybridization and Theoretical Insight

The curse and blessing of backpropagation through time

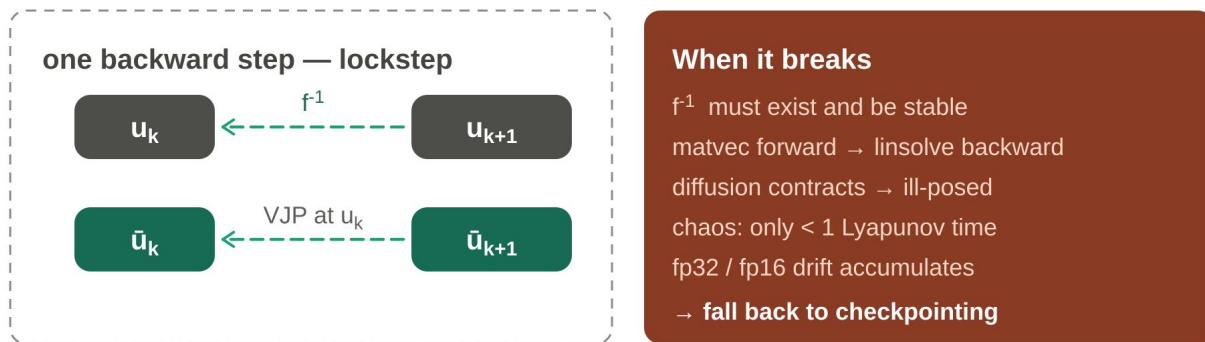
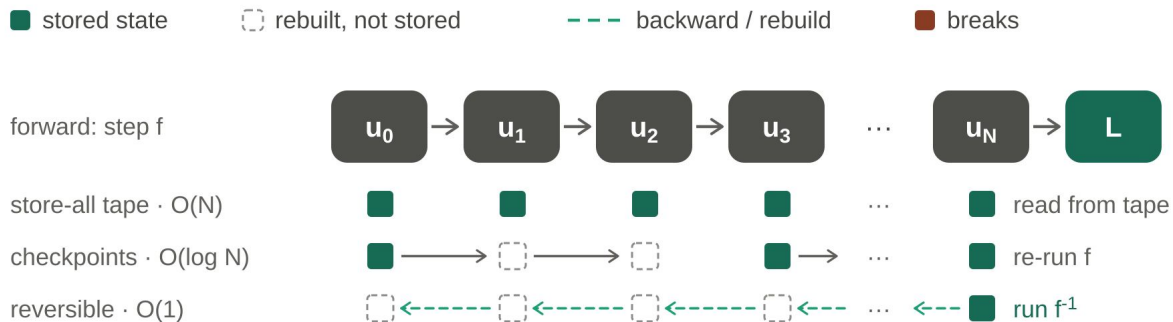


- Compute grows linearly
- Memory grows linearly
- Gradients may explode/vanish

Pragmatically Motivated Gradient Cuts



The $O(1)$ memory backprop through time trick



Linear Solvers in the backward pass

```
from phi.flow import *
velocity = StaggeredGrid(0, x=64, y=64, bounds=Box(x=100, y=100))
smoke = CenteredGrid(0, ZERO_GRADIENT, x=200, y=200, bounds=Box(x=100, y=100))
INFLOW = 0.2 * resample(Sphere(x=50, y=9.5, radius=5), to=smoke, soft=True)
pressure = None

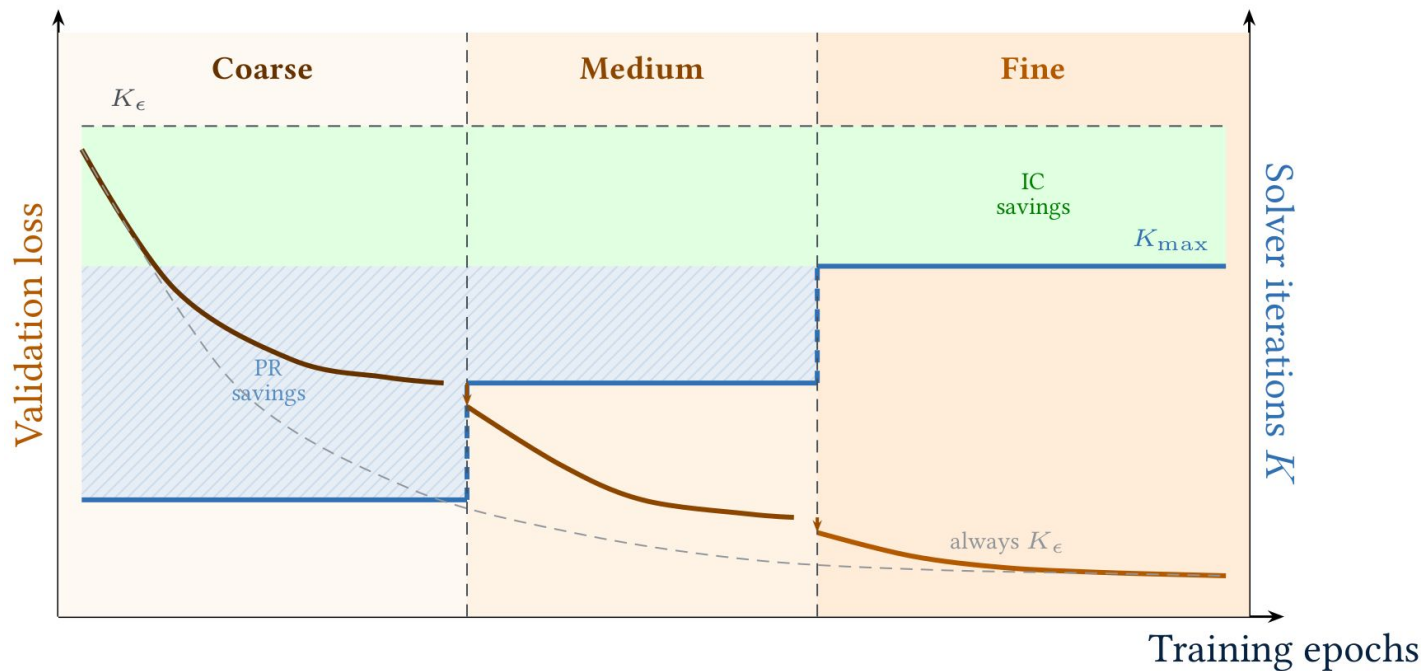
def step(v, s, p, dt=1.):
    s = advect.mac_cormack(s, v, dt) + INFLOW
    buoyancy = resample(s * (0, 0.1), to=v)
    v = advect.semi_lagrangian(v, v, dt) + buoyancy * dt
    ### ---> Linsolve start <---
    v, p = fluid.make_incompressible(v, (), Solve(x0=p))
    ### ---> Linsolve end <---
    return v, s, p

for _ in range(10):
    velocity, smoke, pressure = step(velocity, smoke, pressure)
```

https://github.com/tum-pbs/PhiFlow/blob/c4cec7ba9e62209c7bcfefeba7d87a42fa8a8193/demos/smoke_plume.py



Progressively Refined Differentiable Physics



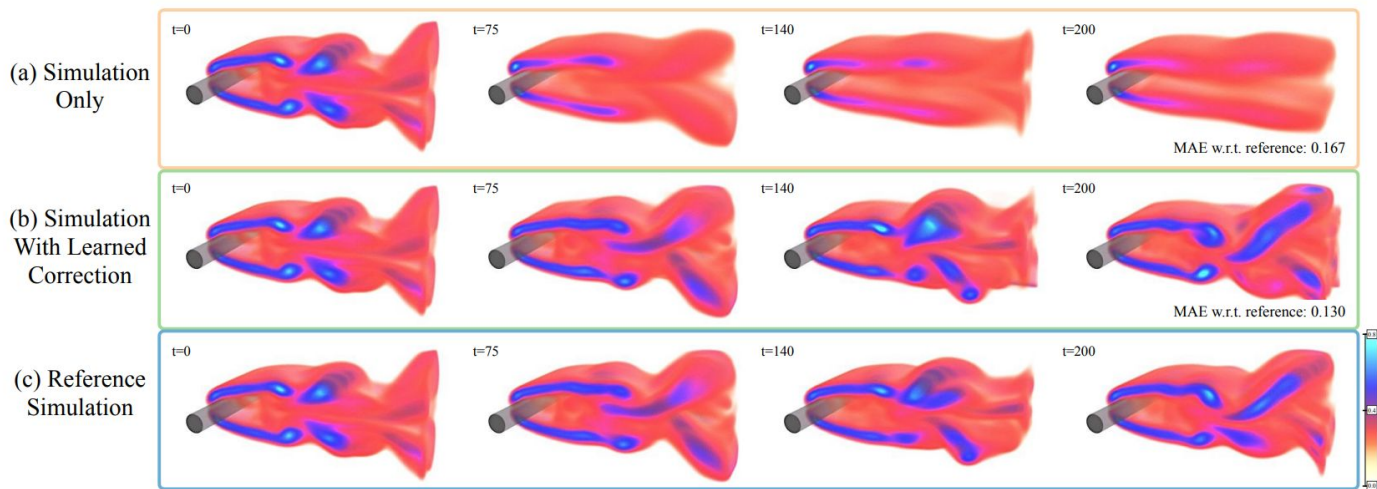
Summary

- Unrolling is important for good performance of autoregressive neural (-hybrid) surrogates applied to time-dependent problems
 - With a small penalty in performance, one can do unrolled training with cutting the bptt and as such not requiring a differentiable solvers
 - The $O(1)$ memory trick can help make unrolled training feasible but its requirements on reversibility are often not met
 - Alternatively, we can find a compromise with checkpointing (-> see exercise)
- Iterative processes within a differentiable solver can be progressively refined during training, reducing the training compute cost

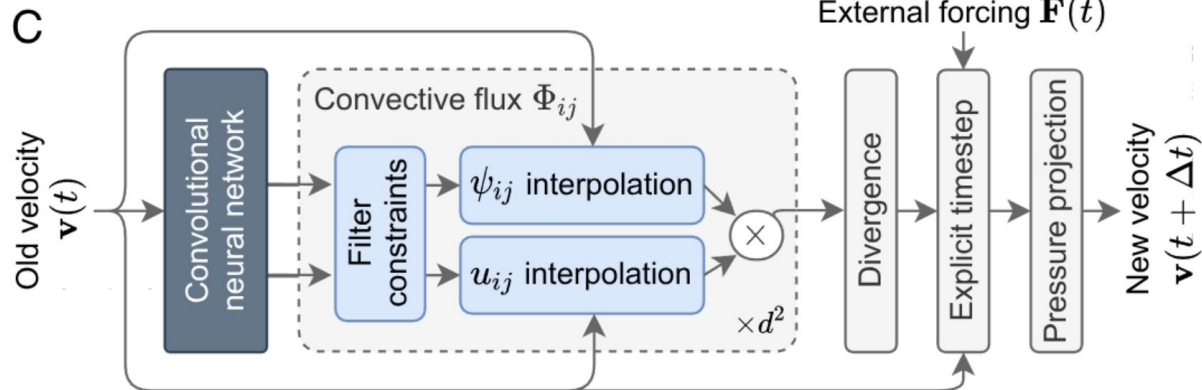
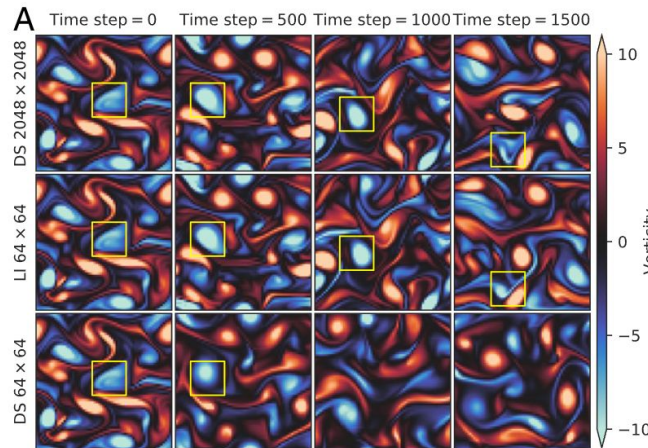
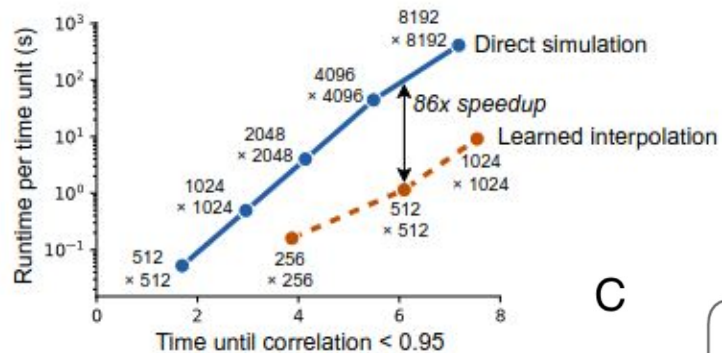
Concrete Case-Studies

Solver-in-the-loop

- Showing that certain physics modes in the simulation will be recovered
 - Later this was then more generalized in that we can recover some high-frequency information (i.e., a learned turbulence model)



ML accelerated CFD

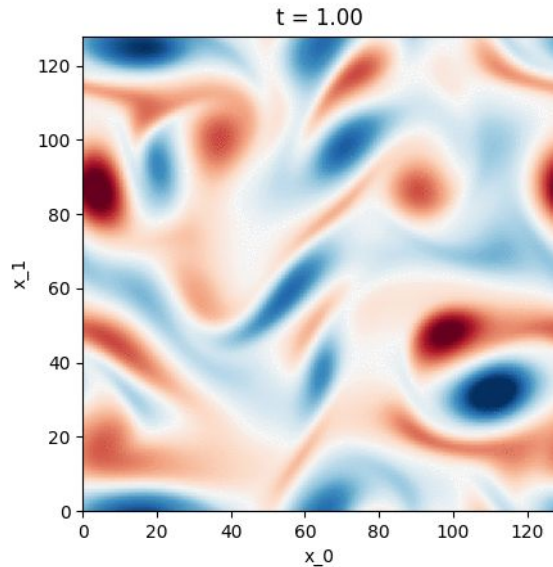


Preparation for the Practical Session

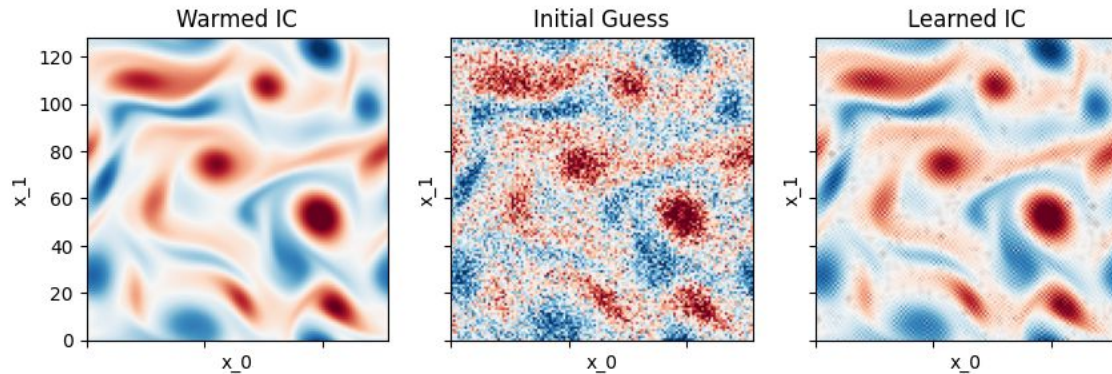
Zeroth Task: Get familiar with the Exponax Solver

Exponax design philosophy:

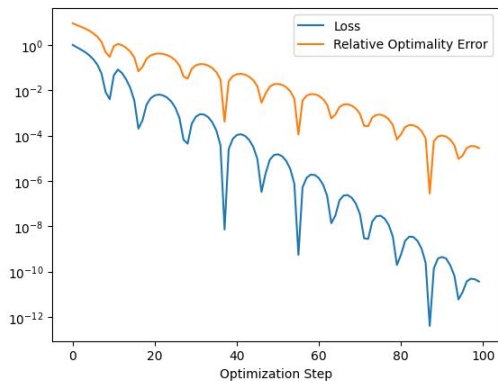
- A numerical simulator is a time stepper which itself is a callable pytree
- Each application advances the (discrete) state by some fixed dt
- As you saw: this is exactly how one could think of neural networks



First Task: Data Assimilation: IC & Diffusivity

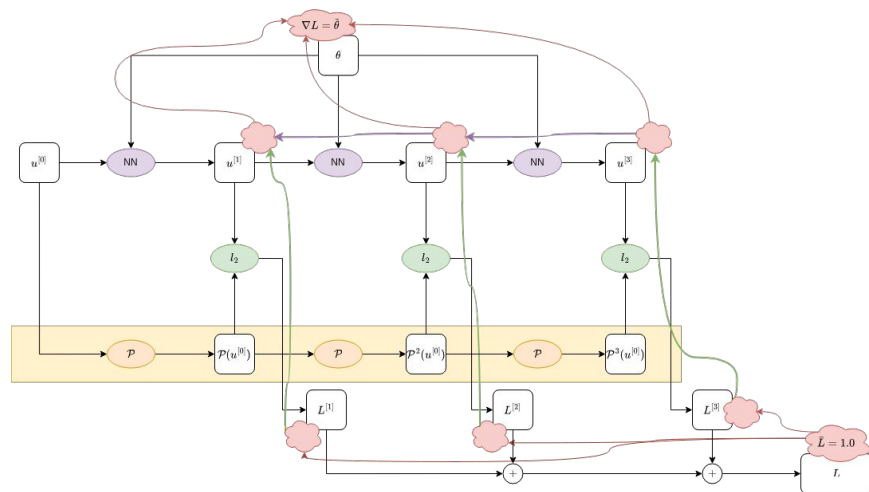
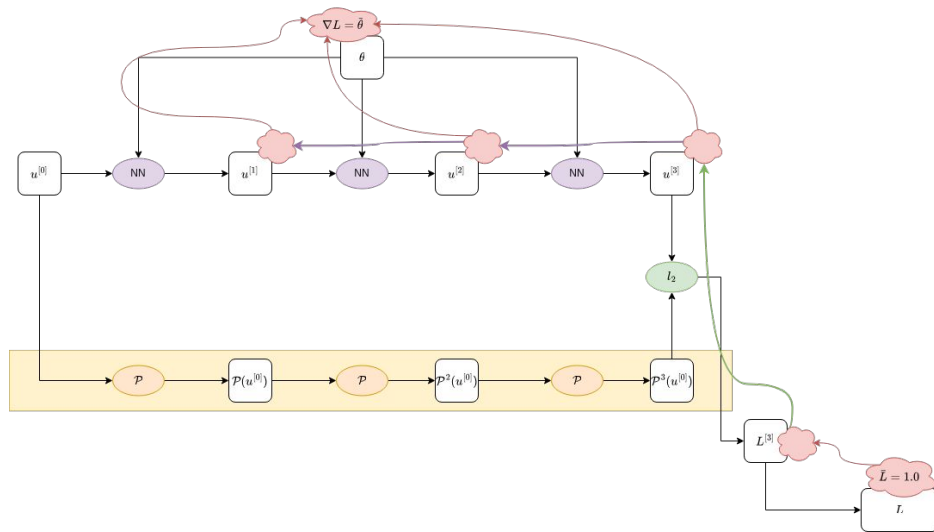


$$u_0^* = \arg \min_{u_0} 1/2 \|\mathcal{P}_h^n(u_0) - y\|^2$$



$$\nu^* = \arg \min_{\nu} 1/2 \sum_{i=1}^n \|\mathcal{P}_h^i(u_0; \nu) - y_i\|^2$$

Final Only vs. Trajectory Aggregation



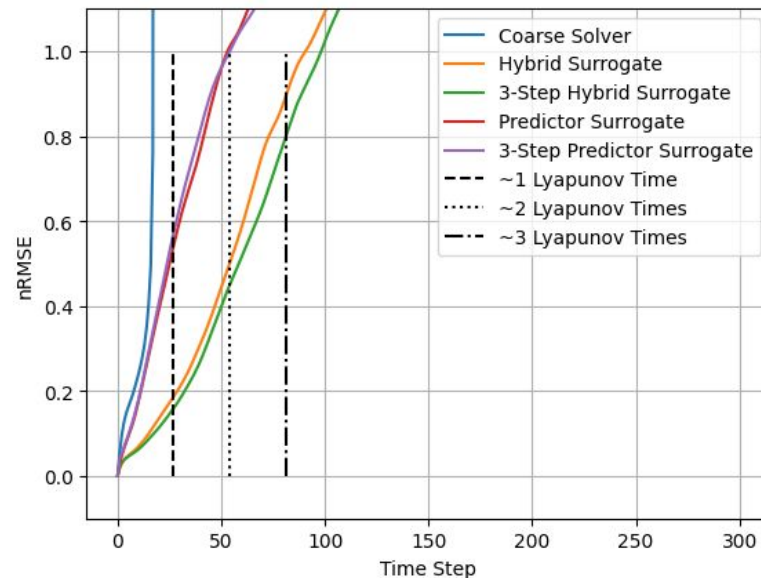
Second Task: Simplified Reproduction of ML-Acc CFD

Machine learning–accelerated computational fluid dynamics

Dmitrii Kochkov^{a,1,2}, Jamie A. Smith^{a,1,2}, Ayya Alieva^a, Qing Wang^a, Michael P. Brenner^{a,b,2}, and Stephan Hoyer^{a,2}

^aGoogle Research, Mountain View, CA 94043; and ^bSchool of Engineering and Applied Sciences, Harvard University, Cambridge, MA 02138

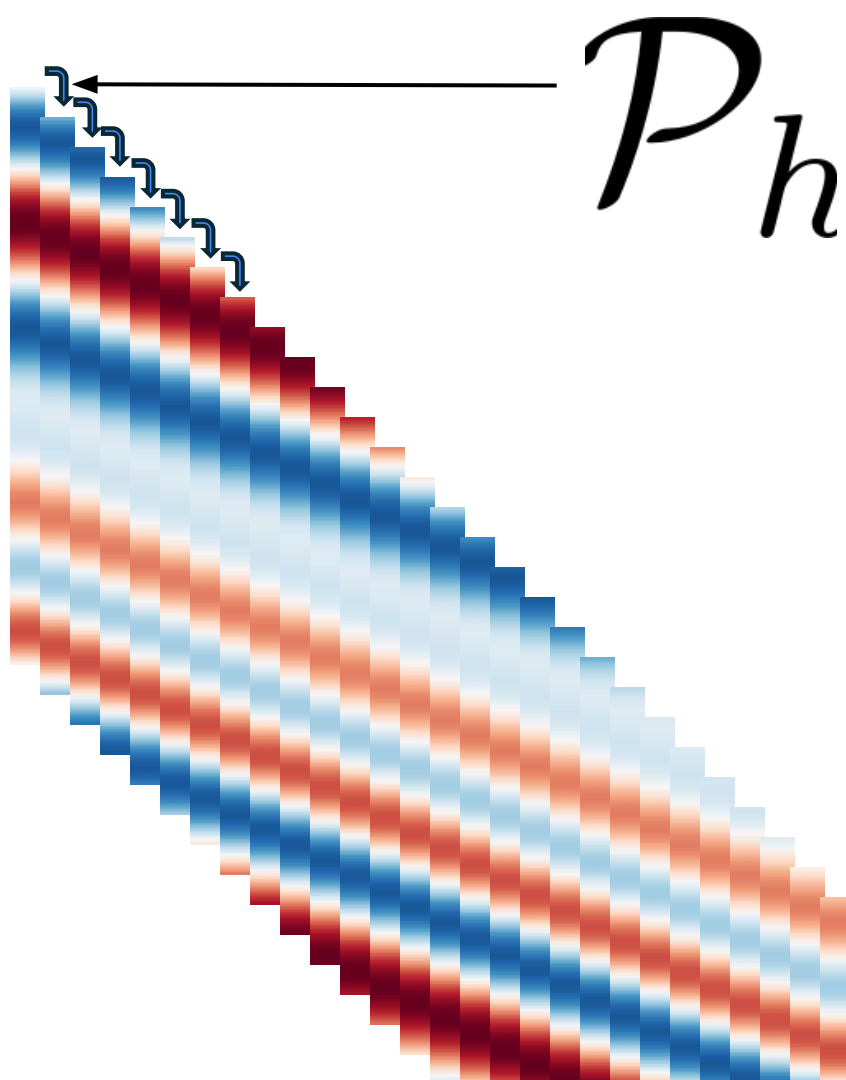
Edited by Andrea L. Bertozzi, University of California, Los Angeles, CA, and approved March 25, 2021 (received for review January 29, 2021)



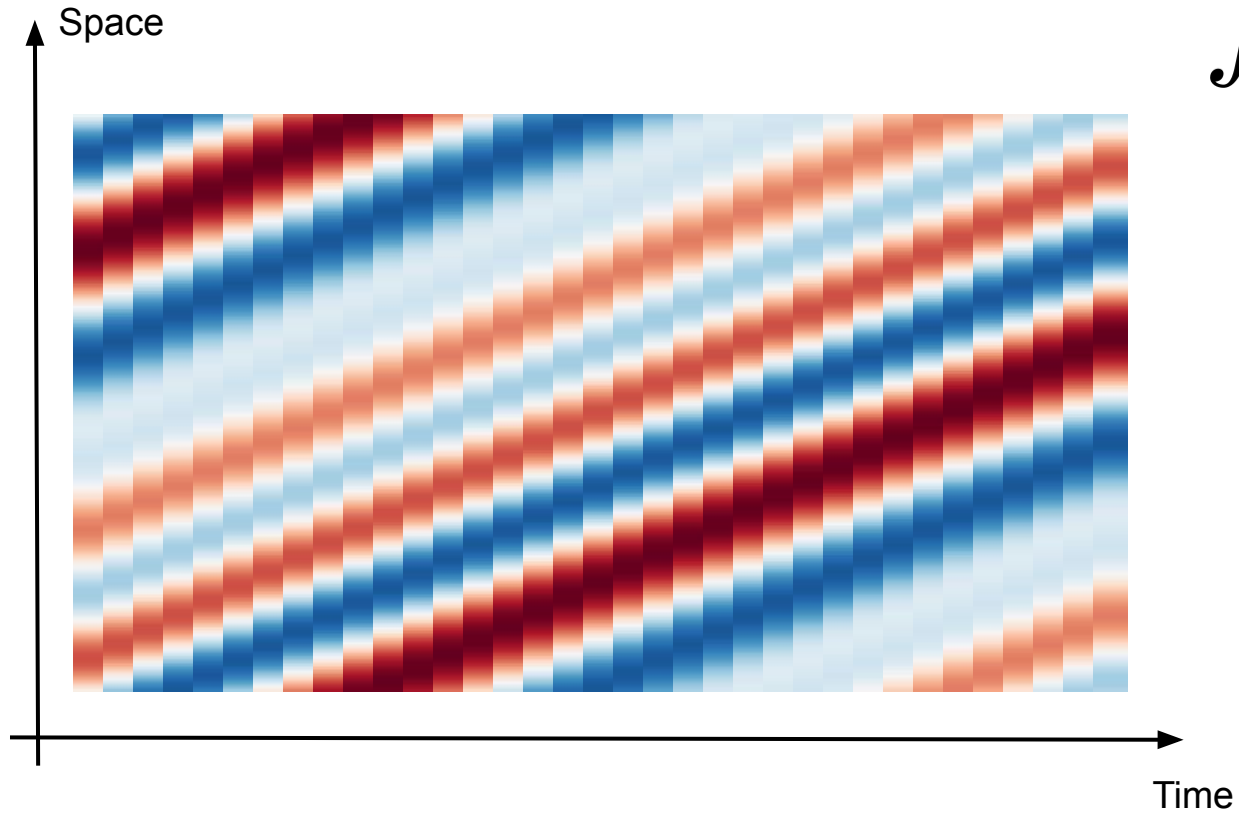
A few caveats

- We are not going to use the operator splitting approach to the Navier-Stokes solver shown before
- This will be based on Exponax: a Fourier pseudo-spectral solver in JAX
 - Technically, this still requires solving a pressure-Poisson problem but we will sidestep the linear solver by employing a fast (and explicit) spectral solve... all done internally in the library
- We will do it a bit looser here:
 - Our goal is not to accelerate but to make it work in the first place
 - We will also not benchmark based on spectral accuracy and decorrelation
 - No learned interpolation but learned correction (since it is less intrusive)

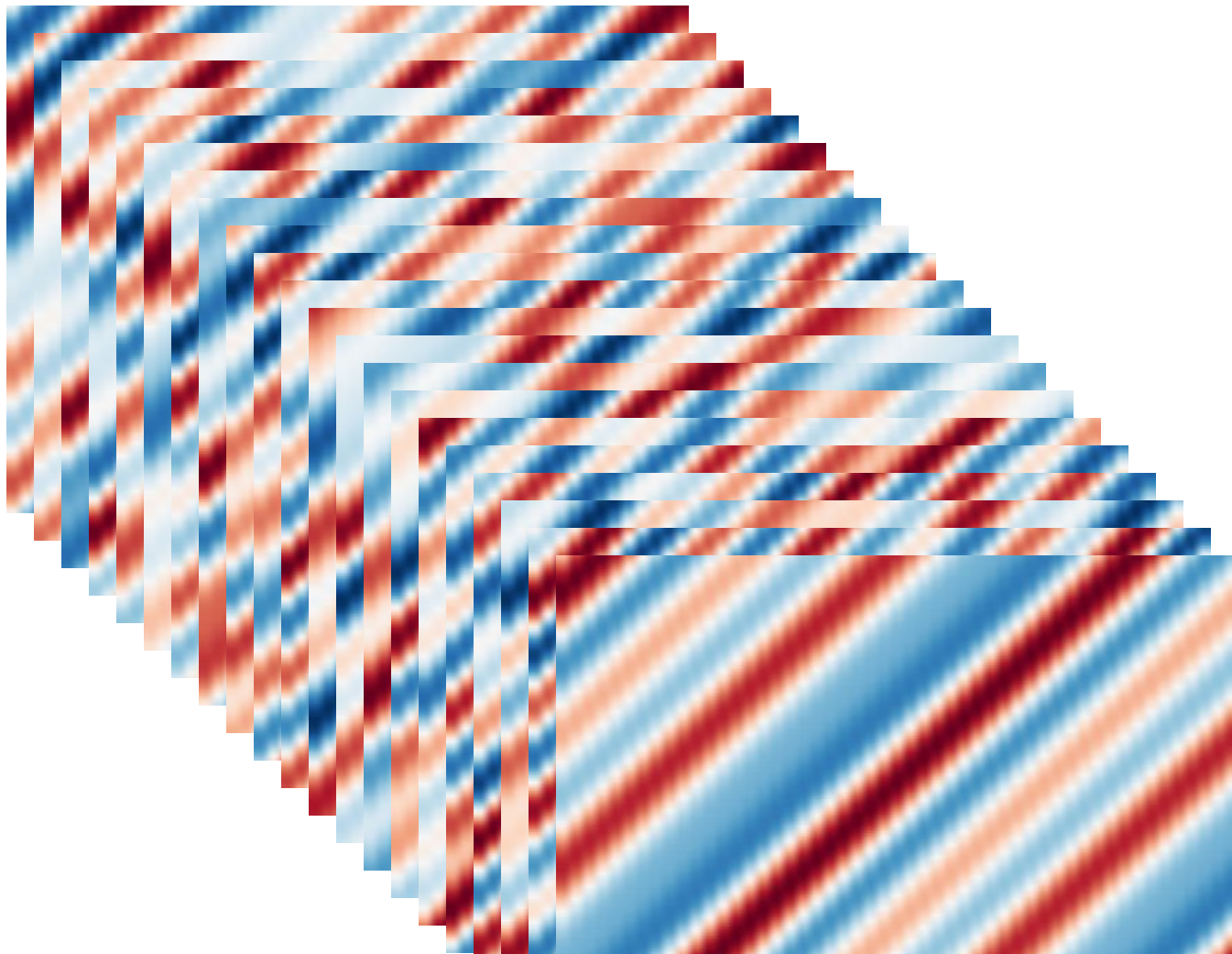
Quick Intro to One-Step Supervised Training

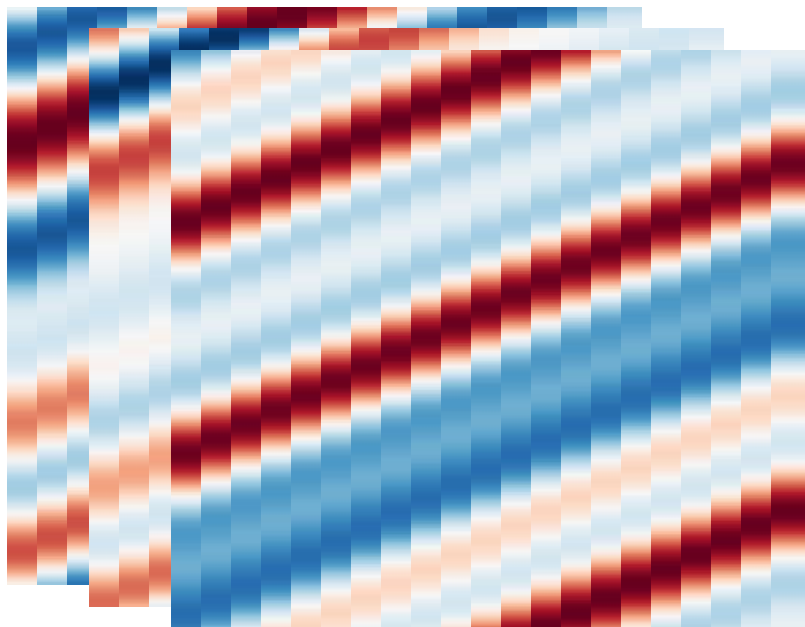


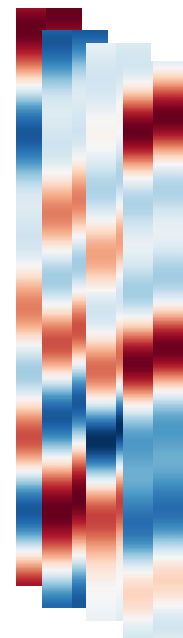
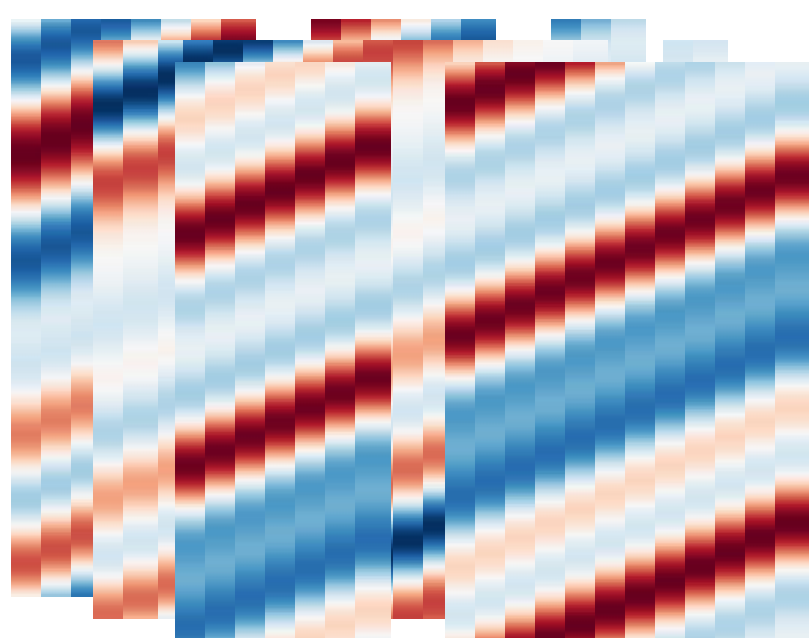
$$f_{\theta} \approx \mathcal{P}_h$$

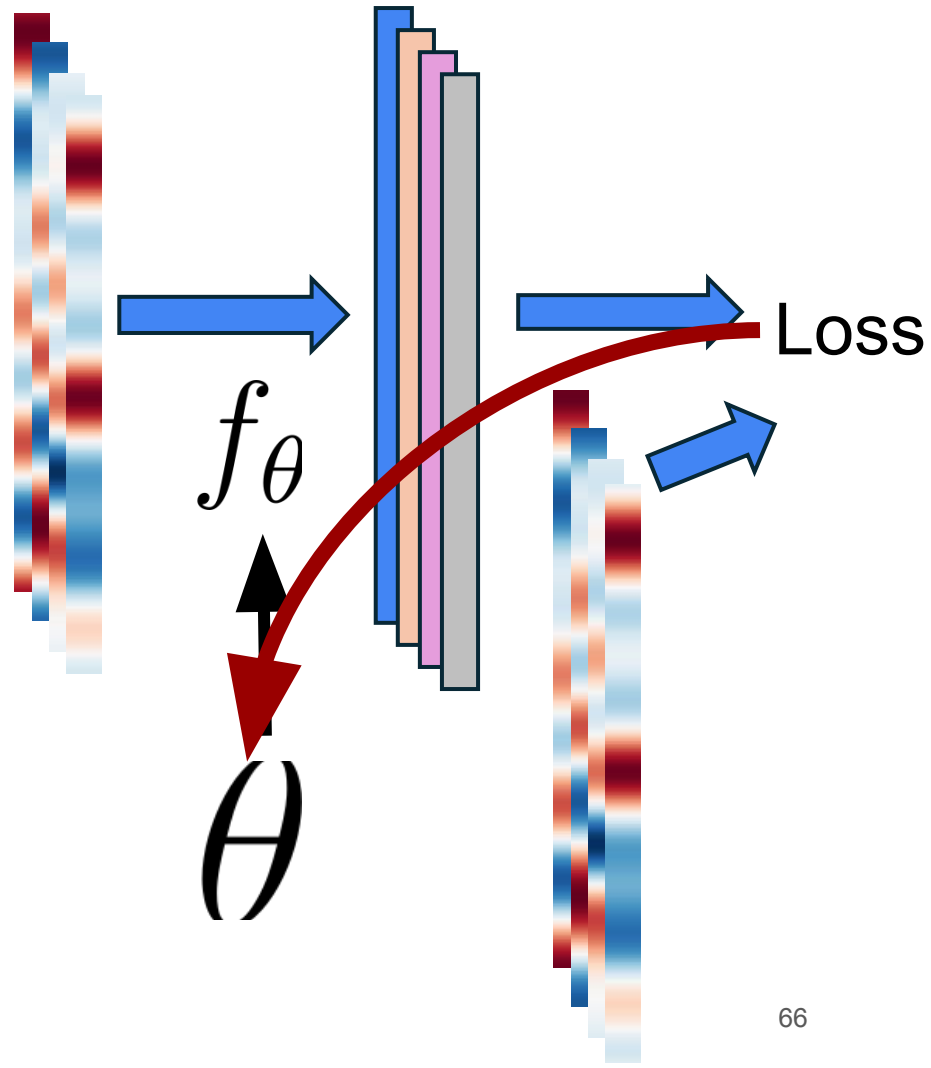
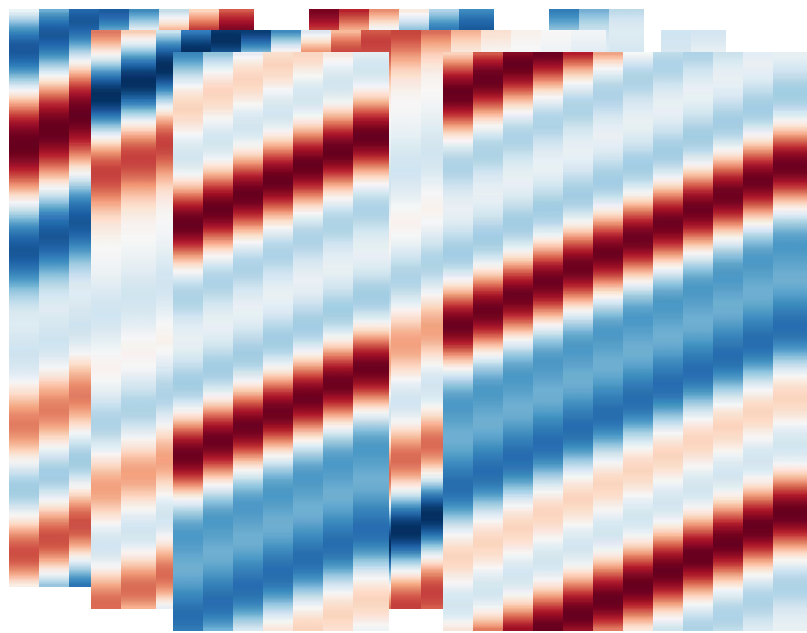












One-Step Training

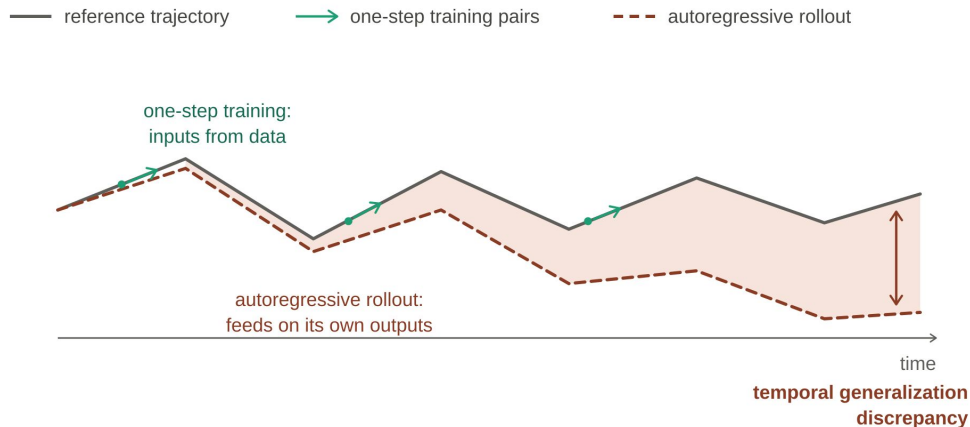


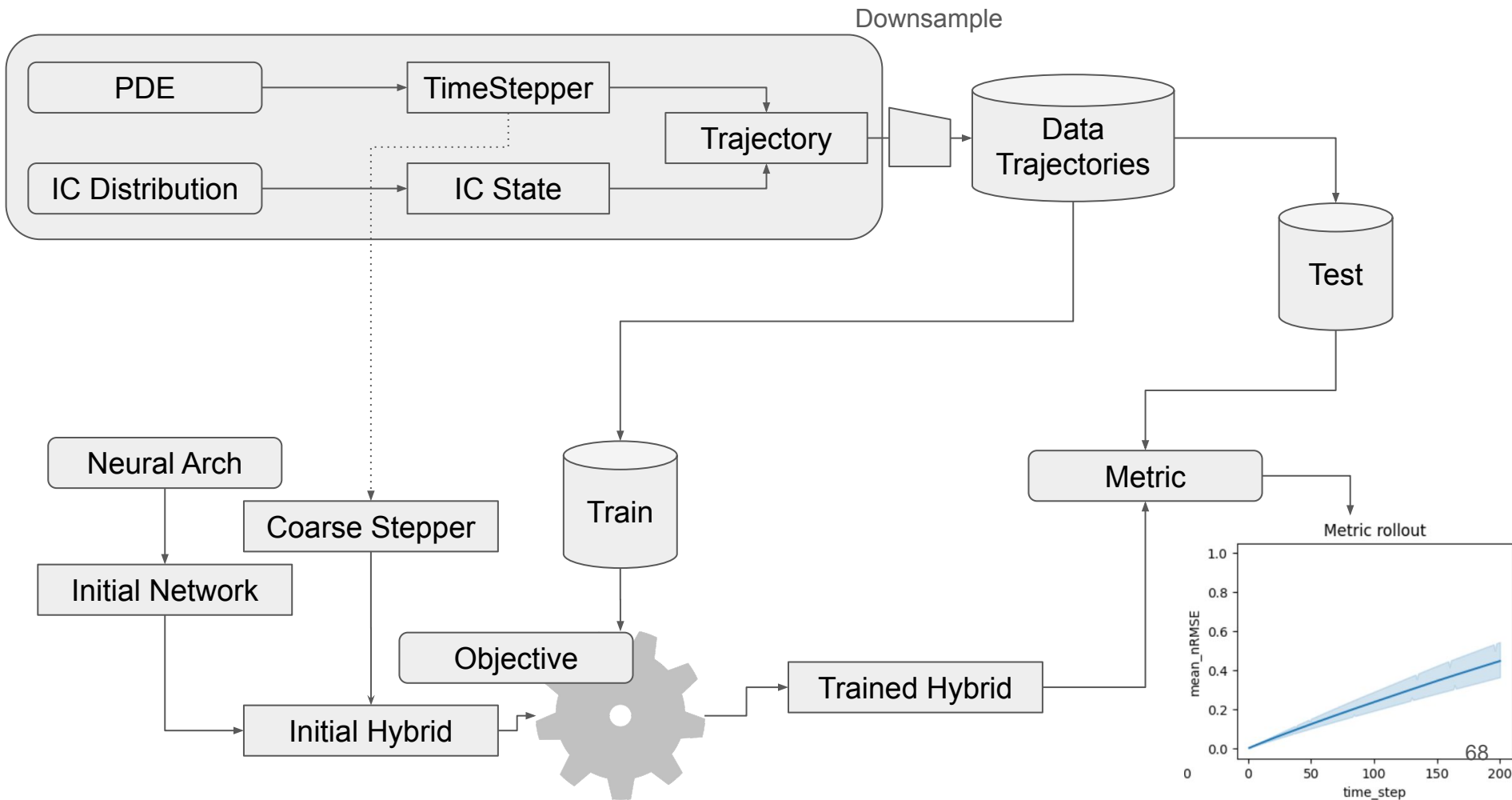
Autoregressive Rollout

Long-Term Accuracy

Long-Term Stability

Temporal Generalization





Introduction to the tooling we will be using

- **PDEquinox**: providing convenience wrappers around Equinox architectures
- **Trainax**: Abstract implementation of various temporal learning methodologies
 - This basically implements the one-step supervised learning for us!
- **Exponax**: A solver for both training data generation (I already did that for you) and for the coarse component we are going hybridize
- **Optax**: Gradient processing in JAX (i.e., the ADAM optimizer)

Please setup the Colab extension for VS Code
(or use Colab Web)

Summary and Learning Objectives

- See similarities between data assimilation and neural-hybrid surrogates
- Understand the impact of backpropagation-through-time
- Get hands-on experience with neural-hybrid surrogates
- Get a feeling for the flexibility achieved when a (fast) solver is available in a machine learning framework

Conclusion

What we did in this talk

- NNs and solvers: same algebra, same autodiff structure
- Differentiable physics vs. its lookalikes (PINNs, Neural ODEs, DEQs, Hamiltonian/Lagrangian NNs)
- All flavors as one thing: a solver in the backward pass (PDE-constrained optimization)
- Examples from the literature: Solver-in-the-loop, ML-accelerated CFD

Summary and Take-Aways

- Same algebra, different parameters:
 - Solvers from first principles vs. neural networks from data
 - Guarantees vs. flexibility
 - Hybridization enables combining the merits
- Differentiable physics = a solver for a concrete ODE/PDE in the backward pass.
 - Parameter learning, data assimilation, full-field correction: one idea at different depths.
- Differentiating a solver isn't free. JAX tooling helps, but spend the effort only when the conditioning pays off
 - Modern research ideas reduce the cost: Hybridization with non-differentiable solvers, adjusting the solver cost over training etc.